Progressive Retrieval Attention for Pretrained Transformers: Sparse Native-K/V Memory with Semantic Routing

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 13, 2026

Abstract

Dense long-context attention couples logical memory size to the key/value (K/V) state active in every transformer operation. PRA-HF separates these quantities: it searches a compact semantic index and materializes only selected layer-native K/V under a fixed attention budget. At the measured pinned Qwen, SmolLM2 Llama-family, and official Gemma 3 1B checkpoints, the verified PRA-off paths leave logits, hidden states, greedy outputs, and the tested cache contracts exactly unchanged.

Controlled positional analysis in companion Paper 1.5 motivates the empirical rule *route semantic state and attend native positional state*. On frozen Qwen3-0.6B, a 262,144-parameter router reaches any-evidence recall 0.831 at 10% and 0.956 at 20% selected parents while returning exact post-RoPE K and native V. Oracle HotpotQA replay is depth dependent: the last 14 layers gain 3.641 nats per gold token, whereas all-layer replay loses 0.766. On a larger identity-disjoint test, frozen routed PRA recovers 11.9% of direct-evidence and 14.1% of feasible full-context sequence-likelihood benefit at 6.56% active K/V. Tiny oracle-trained memory-use adapters approach the clean-memory likelihood ceiling but regress under routed HotpotQA memory, exposing an unresolved robustness gap. The official Gemma path preserves its native local/global organization, and routed K/V raises gold-token log-probability by 0.135 nats although F1 is unchanged in the eight-example causal probe. The shared execution core, thin family adapters, router artifacts, reference lifecycle, accounting, wheel, and Python API make this cross-family boundary directly testable; routing quality and decoded behavioral utility remain model- and domain-dependent.

1 Introduction

Dense long-context attention couples logical memory size to the K/V participating in each transformer operation. Retaining more history therefore increases both stored state and active attention, even when most of that history is irrelevant to the current token. Prompt truncation breaks the coupling by discarding history. Text retrieval narrows the prompt before inference, but then re-encodes the selected text. PRA instead keeps already computed model state outside the active context and restores only selected native K/V.

Paper 1 established bounded sparse native-K/V memory in controlled models [8]. Paper 1.5 separated source coordinates, contextual fidelity, native attention geometry, and semantic routing geometry [7]. This paper asks whether those boundaries can be retrofitted into pretrained Hugging Face decoders without changing their native attention semantics. That requirement is strict. Qwen, Llama-family, and Gemma decoders differ in projection details, RoPE helpers, grouped-query layout, masks, cache protocols, and output paths. An adapter can produce plausible text while violating one of these contracts.

Retrofitting also exposes two distinct interfaces. A routing function R_ϕ answers *what memory should be active?*; a memory-use function M_ψ answers *how should that memory affect frozen computation?* The state best suited to finding a semantically relevant chunk need not be the state a pretrained attention head expects after selection. PRA-HF therefore separates a compact *routing index* from the layer-native *detail K/V* payload, then confines any learned memory-use correction to the PRA-active branch. Paper 1.5 supplies the controlled scientific motivation; this paper tests transfer and implementation in pretrained models. The resulting rule is: **route semantic state; attend native positional state.**

Figure 1 gives the complete data path before the individual contracts are developed. Publication creates a small search representation and a full native payload; only the selected payload enters active attention.

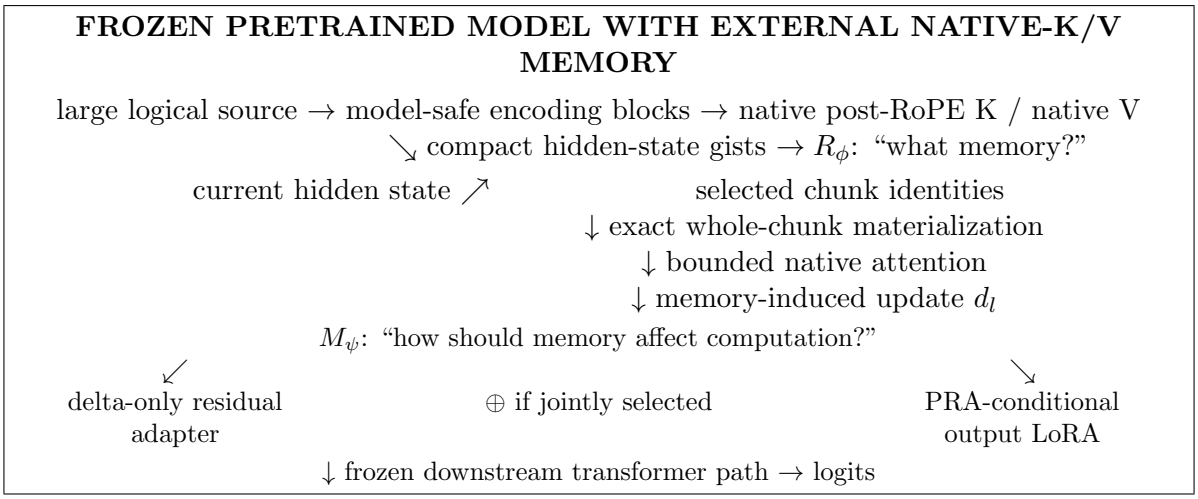

Figure 1: Canonical PRA-HF path and its two learned interfaces. **PRA-HF compresses the search representation, not the selected memory payload.** R_ϕ selects memory from compact semantic state. Native attention consumes exact selected positional K/V. M_ψ is optional, PRA-conditional, and structurally absent when external memory is disabled.

Table 1: Headline findings. The retrofit is exact when disabled, sparse when active, and causally useful at a depth-dependent operating point. Recall definitions and cohorts are stated with each result.

Finding	Result	Why it matters
Exact disabled retrofit	Zero logit and hidden-state difference; identical greedy output	External memory can be added without changing ordinary inference
Semantic routing, native payload	Hidden-state index; selected post-RoPE K and native V	Search features need not replace pretrained attention state
Compact sparse discovery	262,144 parameters; any-evidence recall .831/.956 at 10/20%	A 0.044%-of-model router searches a 0.392%-of-detail index
Depth-dependent causal use	Hotpot oracle $\Delta \log p$: +3.641 last 14; -.766 all 28	More memory exposure is not monotonically better
Bounded-head causal probe	Evidence rank 1; $\Delta \log p = +5.469$; greedy answer wrong	Retrieval, probability change, and decoding are distinct
Third-family portability	Official Gemma 3 1B exact disabled parity; local/global structure preserved; routed $\Delta \log p = +.135$	Shared core survives explicit local/global attention organization
Routed context recovery	11.9% direct / 14.1% full benefit at 6.56% active K/V	Sparse end-to-end use remains positive without memory-use adaptation
Memory-use ceiling	Residual 16: 90.8% direct / 107.3% full at 11.39% K/V	Oracle gains do not survive routing error; residual+LoRA is rejected
Reusable artifact	Python API, CLI, saved routers, metrics, tests, wheel	The measured mechanism is independently exercisable

The paper makes five contributions. First, it defines and tests an exact pretrained-model retrofit for bounded native-K/V memory, including exact disabled-path parity. Second, it separates compact semantic routing state from the selected layer-native positional K/V payload. Third, it measures sparse discovery as a recall–sparsity frontier and shows that a tiny supervised projection improves frozen semantic routing. Fourth, it establishes depth-dependent causal memory use and separates a clean-memory integration ceiling from the routed end-to-end robustness boundary. Fifth, it implements the contract as a shared core, thin Qwen, Llama-family, and Gemma adapters, router artifacts, accounting, a wheel, and a public Python API. The contribution is a working boundary, not a new pretrained model or a solved long-context QA system.

The datasets have different jobs. WikiText-style controls test ordinary language-model integrity and bounded-memory mechanics. QASPER tests one-shot retrieval from a long scientific document. HotpotQA stresses multi-passage discovery and, where direct evidence text helps the checkpoint, provides the cleanest oracle causal probe. It is not a universal pass/fail test for one-shot PRA.

Table 2: Recall metrics answer different questions and are never substituted for one another.

Metric	Meaning
Any-evidence recall	Fraction of examples for which at least one annotated evidence region is selected
Evidence-identity recall	Fraction of all annotated evidence identities recovered
All-evidence recall	Fraction of examples for which every required annotated evidence identity is selected

2 PRA-HF in 60 Seconds

A long source is encoded in blocks that fit the host model. Publication keeps compact semantic gists in a routing index and stores exact layer-native post-RoPE keys and native values as the detail payload, which may remain on CPU. A query searches only the compact index. Selected chunk identities pass through a hard budget, and only their native K/V moves to the accelerator for bounded attention with the direct sequence.

This division answers the most important implementation question. PRA does *not* attend to the gist. The gist is an address label, analogous to a compact catalog entry. Once an address is chosen, attention reads the original native K/V payload. Consequently, routing can use a semantic hidden state without asking a pretrained attention head to consume an unfamiliar representation. Likewise, positional correction is a routing concern only when positional keys are used as gists; the selected attention payload always retains its original native rotary encoding.

The budget distinguishes logical availability from physical activation. A reference may contain thousands of cached tokens, while one operation materializes only B selected tokens plus the direct sequence and a small safety reserve. The mechanism therefore bounds active attention even when the logical source is longer. It does not make storage free: full layer-native detail K/V still occupies CPU memory unless a separate compression or tiering method is added.

Table 3: Five stages of a memory-augmented prediction. Success at one stage does not imply success at the next.

Layer	Question	Present status	Evidence
Transport	Can external native K/V be stored and materialized correctly?	Yes	Exact disabled parity; native GQA K/V; bounded CUDA probes
Discovery	Can relevant memory be selected sparsely?	Strong, domain-dependent	.831 any-evidence recall at 10%; weak cross-domain transfer
Integration	Does selected memory change the pretrained prediction usefully?	Yes, depth-dependent	Hotpot oracle last-half +3.641 nats/token; all-layer replay harmful
Adaptation	Can tiny M_ψ parameters close the context gap?	Oracle only	Residual 16 recovers 90.8% direct benefit; routed frozen PRA remains better
Decoding	Does the intervention reliably change generated answers?	Not yet	Oracle residual EM .25; routed adaptation EM zero

3 Minimal PRA Recap

For layer ℓ, PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks. Materialized memory precedes the direct K/V sequence under one softmax:

$$\mathrm{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \mathrm{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M \right) [V_M; V_D]. \tag{1}$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization [8]. Paper 1.5 established that source position, contextual fragmentation, pooled native-attention geometry, semantic relevance, and downstream use are distinct [7]. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer's actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

4 Exact Retrofitting: Shared Core, Thin Adapters

4.1 Two representations, two jobs

PRA represents a cached chunk as

$$\mathcal{M}_c = (R_c, K_c, V_c), \tag{2}$$

where R_c is a compact routing representation and (K_c, V_c) is the payload used after selection. The objects need not inhabit the same feature space. The canonical HF configuration in this paper sets R_c to one or more gists of the hidden state entering the selected attention block, while K_c is that block's native post-RoPE key tensor and V_c its native value tensor. Token-level hidden states are transient during publication; only compact gists and detail K/V persist.

This split follows the specialization chain

$$h_{\text{in}} \longrightarrow W_K h_{\text{in}} \longrightarrow \text{RoPE}(W_K h_{\text{in}}). \tag{3}$$

The first representation retains broad contextual information, the second is an attention-address representation, and the third conditions that address on token position. Native token attention needs the third. Chunk retrieval may prefer the first. The routing query therefore uses the final attention-input hidden state and is compared only with hidden-state gists. It is never compared directly with Q_{post}. This is an empirical default for the tested Qwen checkpoint, not a claim that hidden states are universally optimal routers.

4.2 Shared execution core

The shared core divides the former monolithic attention path into explicit stages. Query preparation selects the newest valid state and maps GQA query groups into native K/V routing width. Exact packed-index search returns parent identities. The materialization budget then enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native,max}}. \tag{4}$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family's existing local causal mask. The controlled model and HF adapters now call this same core.

4.3 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The Qwen adapter has 272 physical lines (248 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3's Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

The Gemma adapter is narrower. It reuses the same projection and capture path but rejects any layer whose native `is_sliding` flag is true. The wrapper republishes this metadata because `Gemma3DecoderLayer` chooses local versus global rotary embeddings before calling its attention child. Local layers therefore retain their 512-token window, local RoPE base, mask, and hybrid-cache behavior. At global layers, the adapter invokes Gemma's installed eager kernel with its attention-logit soft cap and leaves the native one-head MQA payload unexpanded in storage. No Gemma branch was added to routing, budgeting, materialization, or reference publication.

This organization suggests a compatible hybrid pattern: native local computation plus PRA-enabled global integration. More cautiously, the host model's learned attention-scope structure provides a natural prior for external-memory placement. This is consistent with the Qwen result that sparse external memory should not be injected indiscriminately, but it does not establish a universal placement rule or show that the measured Gemma placement is optimal.

The exact target is `google/gemma-3-1b-it` at revision `dcc83ea841ab6100d6b47a070329e1ba4cf78752`. Its documented contract is 26 layers, four query heads, one K/V head, width 256 per head, and a five-local/one-global periodic schedule; global layers are 5, 11, 17, and 23 under zero-based indexing. Tests instantiate Transformers' real Gemma 3 classes with the same hybrid semantics and require bit-exact disabled logits, hidden states, greedy tokens, and every cache tensor. The official checkpoint passes those gates. Reference payloads remain one-head native MQA on CPU. Architecturally, layers 5, 11, 17, and 23 are global-attention capable; the routed experiment configures layers 17 and 23, those same two layers actually consume selected memory, and layer 23 provides routing state. A 384-token logical prompt is processed through 128-token bounded operations with zero native-limit violations.

4.4 Qwen native semantics

4.4.1 Projection and RoPE

The adapter calls the wrapped module's `q_proj`, `k_proj`, and `v_proj`. Qwen3's existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V. No independent RoPE implementation is introduced.

4.4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. Key-space routing baselines average each pair of query heads sharing one K/V head, producing a vector in the same 8×128 space as a flattened key gist. The canonical hidden-state router instead uses one d_{model} vector and does not alter payload head layout. K/V expansion occurs inside Qwen's eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6}.

4.4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module's HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test

with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Evaluation Design

Table 4 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pre-trained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Table 4: Pinned Qwen checkpoint and PRA placement.

Checkpoint	`Qwen/Qwen3-0.6B`
Revision	`c1899de289a04d12100db370d81485cdf75e47ca`
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10^6 / 40,960
Initial PRA placement	layer 27 only
Depth-study instrumentation / selected band	all 28 layers / layers 20–27
Initial gist / routing policy	mean, one gist/chunk, exact top-k
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

6 Exactness and Bounded Execution

6.1 Disabled-path parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 5 reports exact results for the five-token prompt "The capital of Portugal is" and four-token greedy decode.

Table 5: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

6.2 Native-K/V reference transport

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures $[1, 8, 22, 128]$ K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains "Lisbon", but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

6.3 Implicit head and bounded long prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#__head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

6.4 Transport does not establish discovery

The transport tests intentionally do not establish semantic selection. A diagnostic with one fixed HotpotQA example [10] and one QASPER example [2] shows why. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 6: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA's expected answer is "yes"; every condition begins with "No". QASPER's expected answer is "no"; dense and PRA generations begin with "No", while truncation begins with "Yes". However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The diagnostic therefore separates a functioning K/V transport path from an unsuitable routing representation; it does not grade the architecture as a QA system.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317,

while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

7 Semantic Routing, Native Attention

Controlled experiments in Paper 1.5 show that increasingly accurate approximations of native Q/K geometry do not necessarily improve semantic evidence retrieval: centered $G = 1 \rightarrow 8$ raises Q/K rank fidelity from .794 to .963 while semantic AUC remains .6510–.6516, whereas a learned 32-D relevance geometry reaches .971 AUC with only .178 Q/K fidelity [7]. We therefore treat the routing representation and native attention payload as separate interfaces and ask whether this separation transfers to an unmodified pretrained Qwen model.

Controlled principle from Paper 1.5

native positional-attention geometry $\neq$ semantic routing geometry

$\Downarrow$ pretrained transfer

attention-input hidden query/gist $\longrightarrow$ tiny learned relevance projection

$\downarrow$ selected parent identities

$\downarrow$ exact layer-native post-RoPE K / native V

$\downarrow$ pretrained native attention

Figure 2: **Route semantic state; attend native positional state.** Paper 1.5 motivates the interface separation; Paper 2 tests it in pretrained models. Routing returns identities, while the attention path materializes the host layer's exact native K/V.

The code makes this independence executable. Routing returns stable *selected chunk identities*; a separate core entry point applies the common budget and materializes their payload. A fixed-selection test supplies the same identity set with ordinary-router and oracle metadata. It obtains bit-identical post-RoPE K, identical V, and exactly identical native attention output. Router choice can change the set, but it cannot silently change what a fixed set means to attention.

7.1 Pretrained transfer: what should represent a chunk?

The transfer test compares three parameter-free summaries on 16 matched examples, eight from HotpotQA and eight from QASPER. All use 32-token parents and exact top-k search. Post-RoPE key means retain explicit positional structure; pre-RoPE key means remove that phase from the index; attention-input hidden means retain broader contextual state. The learned row uses the separate 32-identity held-out cohort and five router seeds described in the next section. It is included to show the resulting design progression, not as a paired fourth row. In every condition, a fixed parent identity materializes the same post-RoPE native K and native V.

Table 7: Qwen routing-representation transfer. The first three rows use 16 matched diagnostic examples; the learned row uses 32 held-out identities and reports the five-seed 95% interval at R@3. Score-position r is correlation with normalized source position, and recall is any-evidence recall.

Routing state	Position r	R@3	R@8	R@16	Interpretation
Post-RoPE key mean	.652	.125	.250	.313	Strong positional structure; weak semantic index
Pre-RoPE key mean	.009	.313	.563	.750	Explicit phase removed; incomplete relevance geometry
Hidden-state mean	-.021	.438	.500	.750	Best zero-shot low-budget recall; contextual state
Learned asymmetric-128	-.026	$.631 \pm .069$	.806	.963	Task-specific relevance geometry

The controlled Paper 1.5 geometry separation transfers to a pretrained transformer. Pooled post-RoPE K retains strong positional structure and is a poor semantic routing index in Qwen. Removing explicit RoPE phase nearly eliminates measured position bias but does not itself define the best relevance geometry. Attention-input hidden state is independent of the subsequent K rotation and gives the strongest zero-shot low-budget result. This progression motivates the 128-D router rather than another positional repair.

A centered-RoPE Qwen control reproduces the same dissociation between native-Q/K fidelity and semantic retrieval. Its G sweep, implementation details, parameter-free recall curves, and segment, token, and query ablations are reported in Appendix A. The main result is the transferred interface boundary, not a new RoPE derivation.

8 Sparse Discovery with a Tiny Learned Router

Parameter-free cosine similarity asks the pretrained hidden space to serve a new retrieval objective without supervision. We instead freeze all 596,049,920 Qwen parameters and learn only two projections,

$$s(q, c) = \mathrm{norm}(W_q h_q)^\top \mathrm{norm}(W_c g_c), \tag{5}$$

where h_q is the last query state and g_c is one mean hidden-state gist. The selected asymmetric 1024-to-128 linear router contains 262,144 parameters, 0.044% of Qwen. Training uses 48 examples, validation uses 16, and the held-out test uses 32, with no QA-identity overlap. Validation selects an exhaustive pairwise margin objective; five independent seeds measure the selected configuration.

On the held-out cohort, frozen last-state cosine reaches any-evidence R@3/R@8/MRR of 0.469/0.719/0.413; the learned router reaches 0.631 ± 0.069/0.806/0.498. HotpotQA and QASPER R@3 are 0.588 and 0.675. The paired R@3 gain is 0.163 ± 0.069, position bias is negligible ($r = -0.026$), and the matched shuffled-label control collapses to 0.125 R@3. Supervision, rather than low dimension alone, exposes the improved relevance geometry. The router is not yet general: Hotpot-to-QASPER and QASPER-to-Hotpot transfer both reach only 0.213 any-evidence recall@3. Wider projections, sampled negatives, and one hard-negative refresh do not improve the validation-selected result; Appendix A reports these controls.

8.1 Recall–sparsity, not fixed rank, defines the operating point

Any-evidence recall@3 is an intentionally severe diagnostic, but it changes meaning as source length changes. For each example with N_i parents, we also select $k_i(f) = \lceil fN_i \rceil$. Historical artifacts support *any-evidence recall*: whether at least one annotated parent is recovered. Complete rankings support the stricter *evidence-identity recall*: the fraction of all mapped evidence parents recovered. We never substitute one definition for the other.

Table 8: Any-evidence recall versus requested parent fraction. The learned row averages five seeds and 160 seed–example evaluations. Other rows are descriptive historical cohorts.

Mechanism	Any R@5%	Any R@10%	Any R@20%	Any R@30%	AUC_{0--30}
Post-RoPE key mean	0.125	0.125	0.250	0.375	0.190
Pre-RoPE key mean	0.375	0.563	0.938	1.000	0.700
Hidden-state mean	0.438	0.625	0.625	0.813	0.574
Last-state query	0.531	0.719	0.875	0.906	0.724
Learned margin router	**0.675**	**0.831**	**0.956**	**1.000**	**0.835**

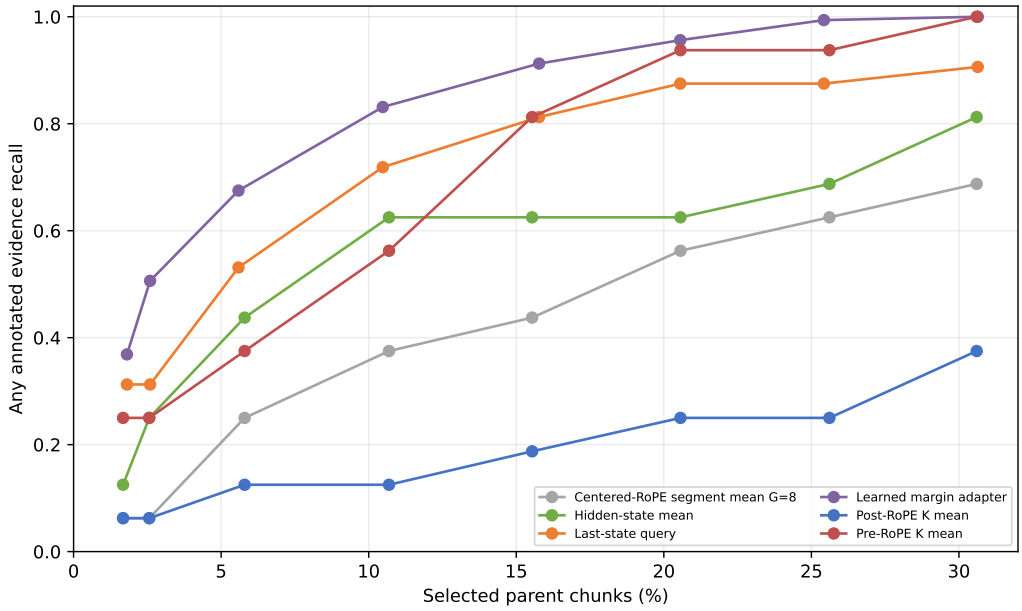

Figure 3: **The tiny learned router moves the recall–sparsity frontier.** Any-evidence recall rises sharply in the 10–20% selected-parent range that fixed-rank evaluation conceals. Cohorts differ for historical baselines, so only the learned result supports uncertainty over seeds.

At requested 10% and 20%, the selected route-once seed recovers 0.356 and 0.442 of annotated evidence identities, while its corresponding any-evidence values are 0.844 and 0.906. These are not the five-seed any-evidence means of 0.831 and 0.956 in Table 8. The seed, cohort, and metric differ. Thus a system can often find one useful passage while still missing other evidence required for a multi-hop answer. This distinction explains why HotpotQA needs a larger realized fraction for 90% any-evidence recall (0.207) than QASPER (0.104).

Scaling must also hold the denominator fixed. In the standalone predecessor, selecting a fixed eight references makes the selected fraction shrink from about 12.5% to 3.1% as memory grows

from 64 to 256 units; apparent coverage consequently falls. Selecting approximately 12.5% instead keeps HotpotQA-derived target coverage at 0.756/0.772/0.762 and QASPER-derived coverage at 0.794/0.803/0.800 for 64/128/256 units. Ranking remains stable at a fixed fraction, while physical materialization grows with that fraction. Sparse quality and active-K/V cost must therefore be reported together.

9 Causal Use Is Depth Dependent

Retrieval recall asks whether annotated evidence was selected. Causal evaluation asks a different question: does the frozen decoder assign more probability to the gold answer after that evidence becomes native K/V? We intervene at both the selection and consumption boundaries. The no-memory condition supplies only the question. Oracle conditions force every 32-token parent intersecting an annotated evidence span. The direct-text control places the same evidence in the ordinary prompt. Four deterministic examples from each dataset make this a mechanism probe rather than an accuracy estimate.

Forced identities enter `prepare_selected_memory`, exactly where ordinary semantic routing ends. Both routes then share thresholding, whole-parent budgeting, native post-RoPE K/V, CPU-to-GPU transfer, GQA layout, source positions, additive masks, and Qwen's eager attention kernel. A selected parent identity is fixed across layers, but its tensor is not: layer ℓ receives $K_c^{(\ell)}, V_c^{(\ell)}$ captured from that layer's own Qwen projections and RoPE path. The 640-token bound reserves 128 direct tokens and permits up to 512 memory tokens per layer. Every oracle parent survives; no budget or native-limit violation occurs, and each reference-encoding call is at most 128 tokens. "Active K/V" is the sum of materialized physical tokens over consuming layers, not the number of distinct source tokens.

Generation-only F1/EM is too coarse for this small frozen checkpoint. The primary paired outcome is therefore teacher-forced mean gold-token log-probability,

$$\Delta_{c,i} = \frac{1}{|y_i|} \log p(y_i \mid q_i, c) - \frac{1}{|y_i|} \log p(y_i \mid q_i, \varnothing), \tag{6}$$

with first-token probability/rank, generated text, attention mass, and answer-position hidden state deltas retained in per-example traces. Four deterministic examples from each dataset are used. This is a mechanism probe, not an accuracy estimate.

HotpotQA supports the integration-depth hypothesis within a late band. The last-one, last-four, last-eight, and last-half interventions move mean log-probability by +0.071, +1.004, +2.311, and +3.641 nats/token. The last eight recover 49.9% of the direct-text effect while using 8/28 layers; the last half recover 78.6% at 1.75 times the last-eight materialization. Generated F1 does not change. The evidence therefore establishes causal likelihood use, not decoded QA success.

This depth dependence is plausible because a residual transformer accumulates context-conditioned updates,

$$h^{(\ell+1)} = h^{(\ell)} + \Delta h^{(\ell)}. \tag{7}$$

Memory introduced too late has few opportunities to affect that trajectory. Memory introduced at every layer can interfere before the local question representation has formed. The measured late band lies between those extremes, but the small probe does not establish a universal theory of transformer hierarchy.

QASPER separates perturbation from interpretable utility. It turns positive at the last eight layers (+0.347) but is only +0.072 at the last half, while direct evidence text is negative (−1.791).

Table 9: Oracle consumption-depth sweep. $\Delta \log p$ is paired mean gold-token log-probability relative to no memory. K/V is aggregate physical materialization across active layers. The key comparison is non-monotone: a late band helps on HotpotQA, whereas all-layer injection harms both datasets.

Dataset	Condition	Layers	$\Delta \log p$	F1	K/V tokens	TTFT	Generation
HotpotQA	none	0	.000	.056	0	.140	.737
HotpotQA	oracle, last 1	1	+.071	.056	112	.143	.733
HotpotQA	oracle, last 4	4	+1.004	.056	449	.154	.813
HotpotQA	oracle, last 8	8	+2.311	.056	898	.169	.877
HotpotQA	oracle, last half	14	+3.641	.056	1,572	.193	1.013
HotpotQA	oracle, all	28	-.766	.000	3,143	.245	1.382
HotpotQA	direct evidence text	0	+4.635	.125	0	.198	.986
QASPER	none	0	.000	.125	0	.135	.781
QASPER	oracle, last 1	1	-.091	.125	240	.137	.785
QASPER	oracle, last 4	4	-.161	.125	960	.149	.844
QASPER	oracle, last 8	8	+.347	.125	1,920	.163	.899
QASPER	oracle, last half	14	+.072	.100	3,360	.188	1.081
QASPER	oracle, all	28	-7.492	.000	6,720	.249	1.488
QASPER	direct evidence text	0	-1.791	.000	0	.416	1.090

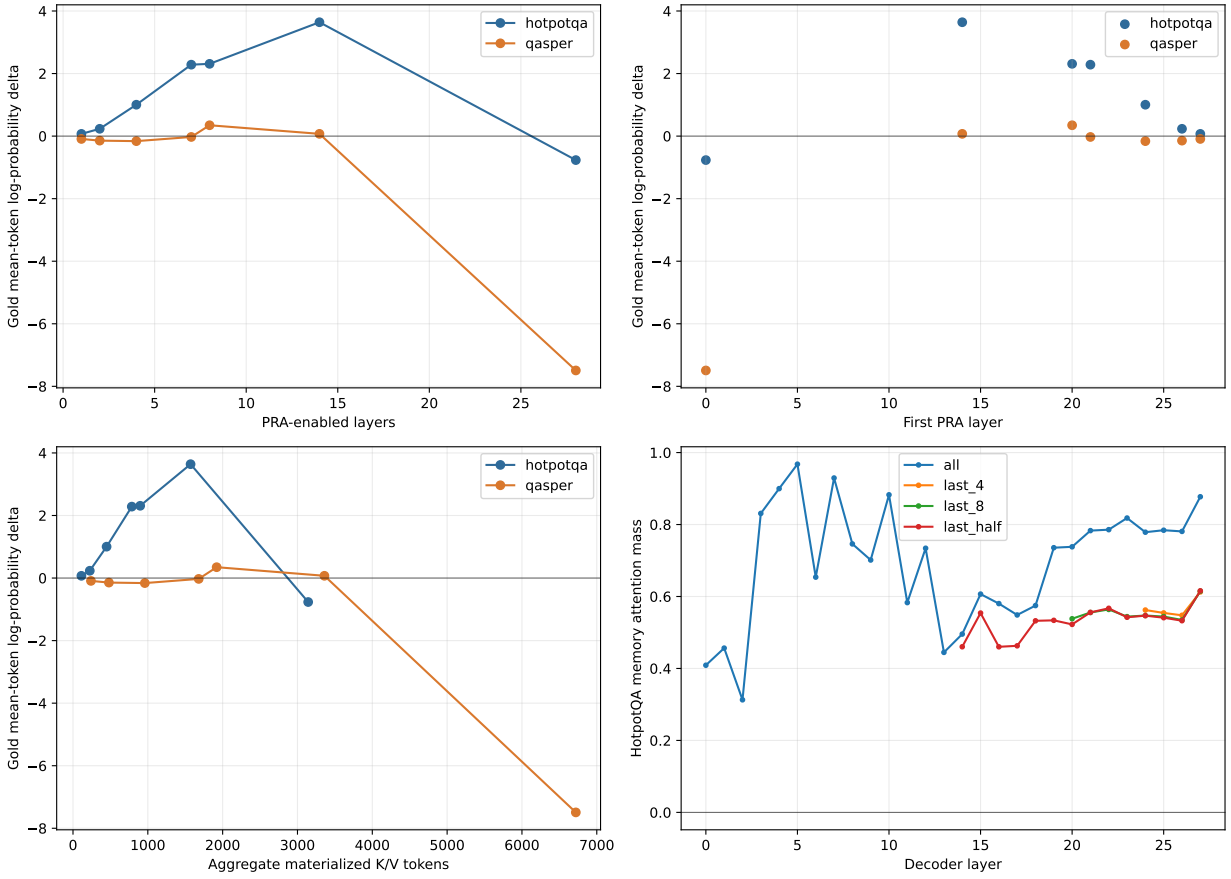

Figure 4: **Retrieved native K/V has a depth-dependent causal effect.** A contiguous late band approaches the direct-text benefit on HotpotQA, whereas all-layer sparse injection is harmful. The lower panels show that additional exposure also increases materialization cost.

That control failure prevents using this subset to grade PRA transport. Attention mass and hidden-state perturbations are retained in the artifact, but neither alone is evidence that a perturbation is useful.

9.1 Placement is not interchangeable

The depth result justifies one placement follow-up. Table 10 fixes either four or seven consuming layers and leaves oracle parent identities, positions, prompts, and per-layer payload sizes unchanged.

Table 10: Oracle placement at fixed memory identities. Equal-count rows have equal aggregate K/V cost, isolating where memory enters the residual trajectory.

Placement	Layers	First layer	$\Delta \log p$	F1
Early contiguous	4	0	-10.573	.000
Middle contiguous	4	12	-.556	.090
Evenly spaced	4	0	-10.398	.000
Late contiguous	4	24	+.421	.090
Every fourth	7	0	-6.090	.000
Late contiguous quarter	7	21	+1.128	.090

This rejects both proposed shortcuts. Earlier insertion does not help by leaving more downstream depth, and every-fourth insertion does not approximate dense late consumption. The likely mechanism is a layer-specific contextualization/topology mismatch: independently encoded reference states are least compatible with early residual computation, whereas a contiguous upper band can repeatedly integrate them after the local question has become semantically formed. This interpretation is mechanistic, not yet a general theorem about decoder depth. All-layer replay is especially informative: more attention opportunities can make the answer worse. The operating point is therefore a band, not "as many PRA layers as possible."

9.2 Route once, consume with layer-native payloads

The routed follow-up scores the question once at layer 27, selects three parent IDs, and reuses those identities across either the last 8 or last 14 layers. Every layer resolves the ID to its own post-RoPE K/V; no tensor crosses layer boundaries. Table 11 reports the complete pre-top-k evidence-identity ranking and the aggregate consumption cost.

Table 11: Combined route-once result over eight examples. Recall is evidence-identity recall, the fraction of mapped evidence-parent identities recovered; f_{80} and f_{90} are the first measured parent fractions reaching those levels.

Router	Consumption	ID R@5%	ID R@10%	ID R@20%	ID R@30%	f_{80}	f_{90}	$\Delta \log p$	F1	K/V
Learned margin	last 8	.264	.403	.521	.639	.502	1.000	+.227	.090	768
Learned margin	last half	.264	.403	.521	.639	.502	1.000	+.374	.028	1,344
Raw hidden-state cosine	last 8	.092	.224	.414	.573	1.000	1.000	+.150	.090	768
Raw hidden-state cosine	last half	.092	.224	.414	.573	1.000	1.000	+.568	.090	1,344

The learned router gives the stronger sparse ranking and reaches evidence in 5/8 selected top-3 sets. Last-eight consumption preserves the no-memory F1 and moves likelihood by +0.227; last-half consumption moves it by +0.374 but lowers F1 to .028. Raw cosine has weaker sparse recall yet a larger last-half likelihood shift. This is not a contradiction: a selected non-annotated parent

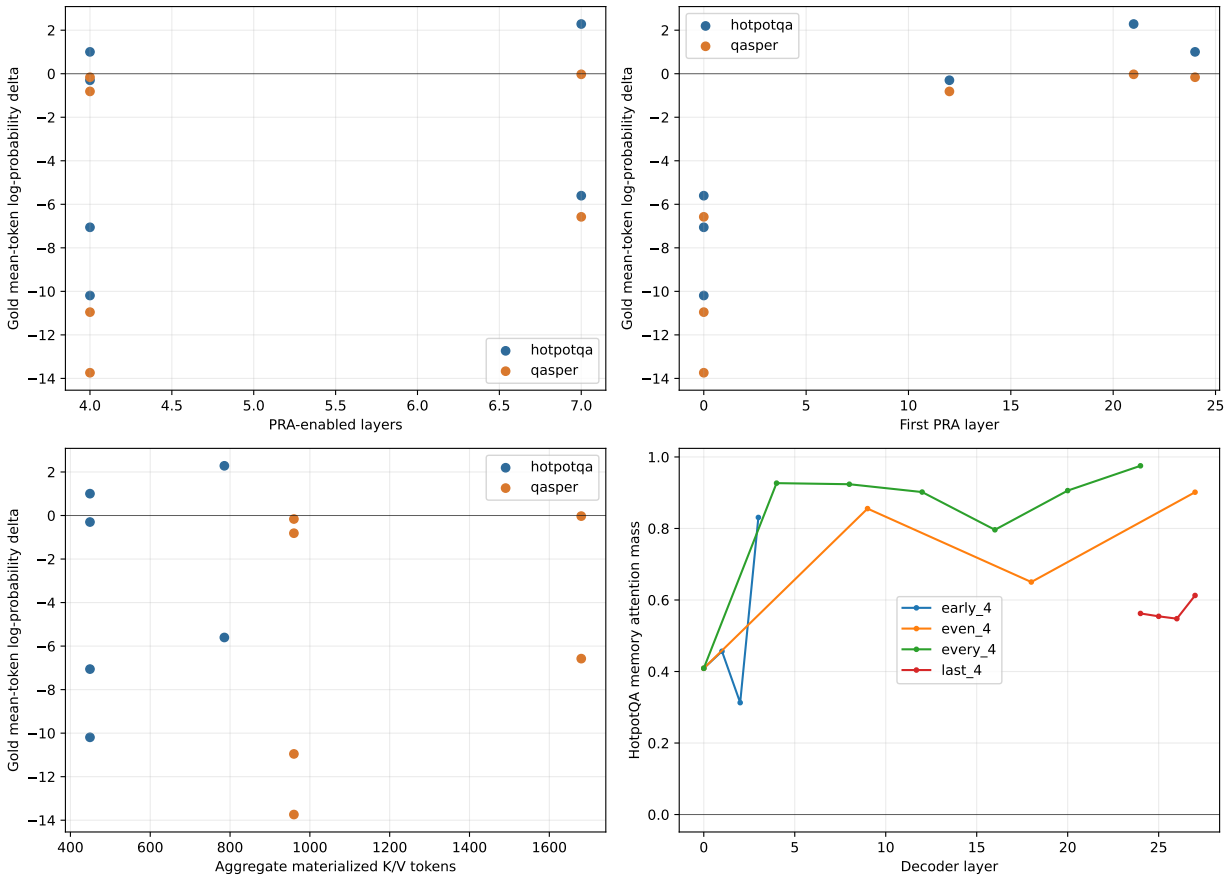

Figure 5: **A contiguous late band is not interchangeable with earlier or sparse placement.** At matched parent identities and per-layer payload, earlier schedules reverse the likelihood effect despite equal K/V cost at equal layer counts.

can still change the answer distribution, and a likelihood movement need not cross the greedy decoding boundary. The learned last-eight condition is the conservative quality/cost operating point. It materializes 768 K/V tokens across layers, increases TTFT from .141 to .167 seconds and generation from .759 to .907 seconds, and adds one .149-second query encoding plus .021-second exact route. Its layer-27 learned index averages 45,184 bytes; storing projected gists for all 28 instrumented layers averages 1.27 MB, versus 322.7 MB of resident detail K/V. These eager single-example timings characterize this implementation only.

9.3 Retrieval, causal influence, and decoding are distinct

The earlier single-layer `#__head` run established bounded displacement but generated no correct answer. We repeat that probe with the learned router at layer 27 and the selected late-eight consumption band. The 416-token logical prompt displaces 288 tokens, retains a 128-token direct tail with positions 288–415, and indexes nine 32-token parents. The router ranks the evidence parent first and selects it with two neighboring parents. Each layer 20–27 then consumes its own 96-token native K/V payload.

Table 12: Bounded implicit-head confirmation. The router ranks the evidence first and selected memory strongly improves the gold distribution, but the greedy answer remains wrong. Rank is the gold first-token vocabulary rank; K/V is aggregate physical materialization.

Condition	Evidence rank	$\Delta \log p$	First-token rank	K/V	F1	Viol.
No memory	–	.000	94,106	0	.000	0
Learned route, last 8	1	+5.469	1,351	768	.000	0

The intervention raises the two-token answer's mean log-probability from -10.924 to -5.455 and its first-token probability from 3.79×10^{-9} to 1.95×10^{-5}. Greedy output still omits "cobalt." The maximum native operation remains 128 tokens and no violation occurs. Thus one bounded run demonstrates successful discovery, layer-native transport, and strong causal likelihood use together. It also shows why they must be measured separately from decoded task success: a five-order-of-magnitude first-token probability increase need not cross the greedy decision boundary.

9.4 Oracle integration ceiling and routed end-to-end robustness

The depth sweep identifies layers 14–27 as Qwen's strongest measured memory-consumption band. We therefore rerun the complete memory-use ladder at that depth instead of extrapolating from the preliminary late-four study. This experiment fixes the backbone, router, selected native K/V, prompt, and attention budget. It varies only M_ψ, the PRA-active transformation of the memory effect. The residual alternative forms

$$d_l = y_l^{\text{mem}} - y_l^{\text{local}}, \qquad y_l^{\text{res}} = y_l^{\text{mem}} + W_{\text{up},l}\,\text{SiLU}(W_{\text{down},l} d_l + b_l), \tag{8}$$

where the bottleneck width is 16, 32, or 64. The LoRA alternative modifies the native output projection only on a PRA-active call:

$$y_l^{\text{LoRA}} = W_{O,l} z_l + \frac{\alpha}{r} B_l A_l z_l, \qquad r \in \{4, 8\}, \quad \alpha = r. \tag{9}$$

Both up projections start at zero. The required combination adds both corrections after the same native memory attention. Every alternative is structurally bypassed when PRA is inactive.

Thus logits and greedy generation remain bit-exact to vanilla Qwen without relying on a regularizer to approximate parity.

The protocol trains on 12 HotpotQA identities for 32 oracle-memory updates, selects width and rank on four disjoint HotpotQA plus four disjoint QASPER identities, and tests on a further eight identities from each dataset. Seeds 11, 23, 37, 53, and 71 vary initialization and example order. Validation selects residual width 16 and LoRA rank 4; their combination is then trained without consulting test identities. The primary score is the gold sequence log-probability change from the same no-context prompt. Recovery is a ratio of matched cohort means, and is reported only when the direct or full-context denominator is positive and greater than 0.05.

Table 13: Identity-disjoint last-14 adaptation comparison. Parameters count M_ψ only; every row also uses the fixed 262,144-parameter router. H and Q denote HotpotQA and QASPER. O is the clean-memory oracle integration ceiling; R is the routed end-to-end path. Entries are sequence-log-probability changes. ρ_D is HotpotQA direct-evidence recovery for O/R; F1/EM is routed. QASPER recovery is omitted because direct evidence reduces cohort likelihood. All disabled-PRA checks are exact.

Method	M_ψ params	% base	H O	H R	ρ_D O/R	Q O	Q R	H F1/EM	Q F1/EM
Frozen PRA	0	.000	+11.212	+1.901	70.3/11.9	+1.444	+1.377	.000/.000	.073/.000
Residual 16	458,976	.077	**+14.478**	**+1.395**	**90.8**/8.8	+5.067	+4.942	.028/.000	**.205**/.000
Residual 32	917,952	.154	+13.792	-.961	86.5/-6.0	+4.767	+5.108	.017/.000	.175/.000
Residual 64	1,835,904	.308	+12.884	-3.443	80.8/-21.6	+4.458	+5.148	.028/.000	.159/.000
Output LoRA $r = 4$	172,032	.029	+12.191	-4.171	76.5/-26.2	+5.160	+5.542	**.049**/.000	.164/.000
Output LoRA $r = 8$	344,064	.058	+11.057	-6.979	69.4/-43.8	**+5.177**	**+5.589**	.031/.000	.135/.000
Residual 16 + LoRA 4	631,008	.106	+12.035	-5.051	75.5/-31.7	+5.140	+5.583	.022/.000	.171/.000

The larger cohort rejects complementarity. On HotpotQA oracle replay, residual 16 exceeds frozen PRA by 3.266 ± 0.298 sequence log-probability and does so for all five seeds. It recovers 90.8% of the direct-evidence benefit and 107.3% of the feasible full-context benefit while materializing 11.39% of source K/V. Its oracle F1/EM is .314/.250. Yet under learned routing, residual 16 gains only 1.395 versus frozen PRA's 1.901 and recovers 8.8% rather than 11.9% of direct evidence. The paired difference from frozen is $-.506 \pm .523$ and changes direction across seeds.

LoRA and the combination fail more sharply on routed HotpotQA. The combination trails residual 16 by -6.446 ± 0.551 in every seed and trails LoRA by $-.881 \pm 1.280$ without a consistent seed direction. Oracle residual and LoRA therefore do not repair independent mismatches that can be profitably stacked. Training on clean oracle memory instead makes the larger conditional updates brittle to routing errors. The final end-to-end architecture consequently retains frozen last-14 PRA plus R_ϕ; residual 16 is an integration-ceiling instrument, not a default product component.

A plausible interpretation is selection-distribution shift. Adapter training presents clean, annotated memory, whereas deployment presents the router's selected memory and its characteristic omissions and distractors:

$$P_{\text{train}}(M) \approx P_{\text{oracle}}(M) \neq P_{\text{router}}(M) \approx P_{\text{deploy}}(M). \tag{10}$$

The oracle-to-routed reversal is consistent with this mismatch but does not prove it. It does show that the memory consumer and router cannot be assumed to optimize independently. Future memory-use adapters should be trained and validated on the router's actual selection and error distribution, not only on gold memory.

QASPER provides a useful but bounded transfer result. All trained alternatives improve routed sequence likelihood over frozen PRA; LoRA rank 8 reaches +5.589 and residual 16 reaches routed F1 .205. However, direct evidence itself changes sequence likelihood by -1.182 on this cohort,

and no full source fits the 2,048-token control limit. Recovery ratios are therefore undefined, EM remains zero, and these values cannot support a context-benefit claim.

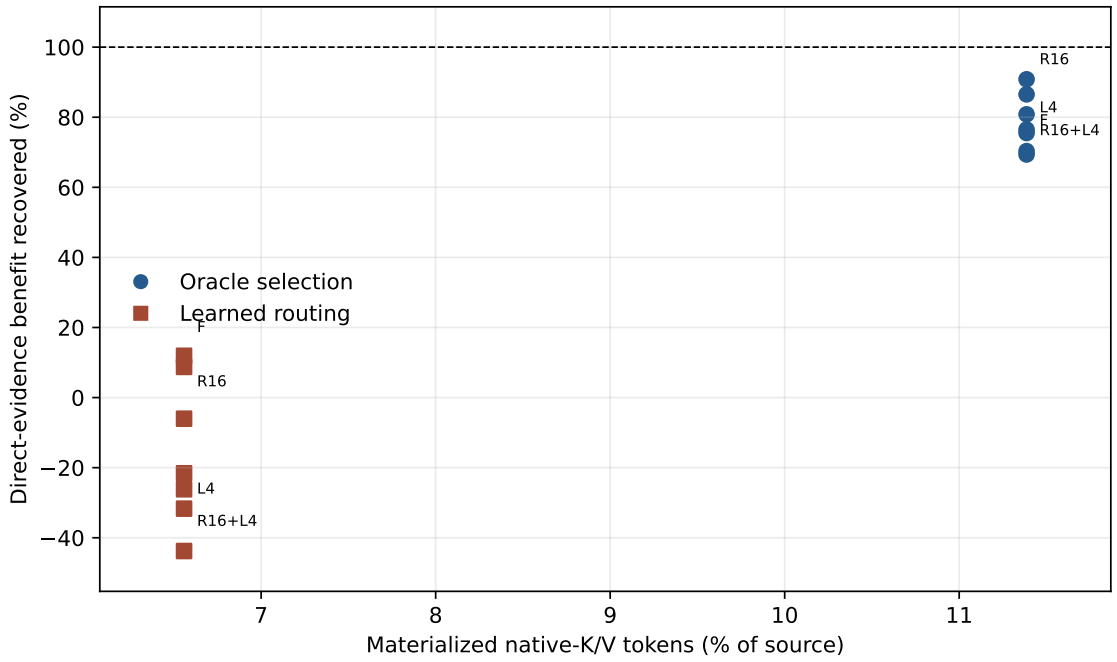

Figure 6: HotpotQA direct-evidence recovery versus physical native-K/V materialization. F, R16, L4, and R16+L4 label the frozen, residual-16, LoRA-4, and combined controls; unlabeled points are the remaining registered widths/ranks. All methods share a fixed memory fraction within a selection mode. Oracle residual 16 raises the ceiling, while every learned adaptation lowers the routed frontier relative to frozen PRA.

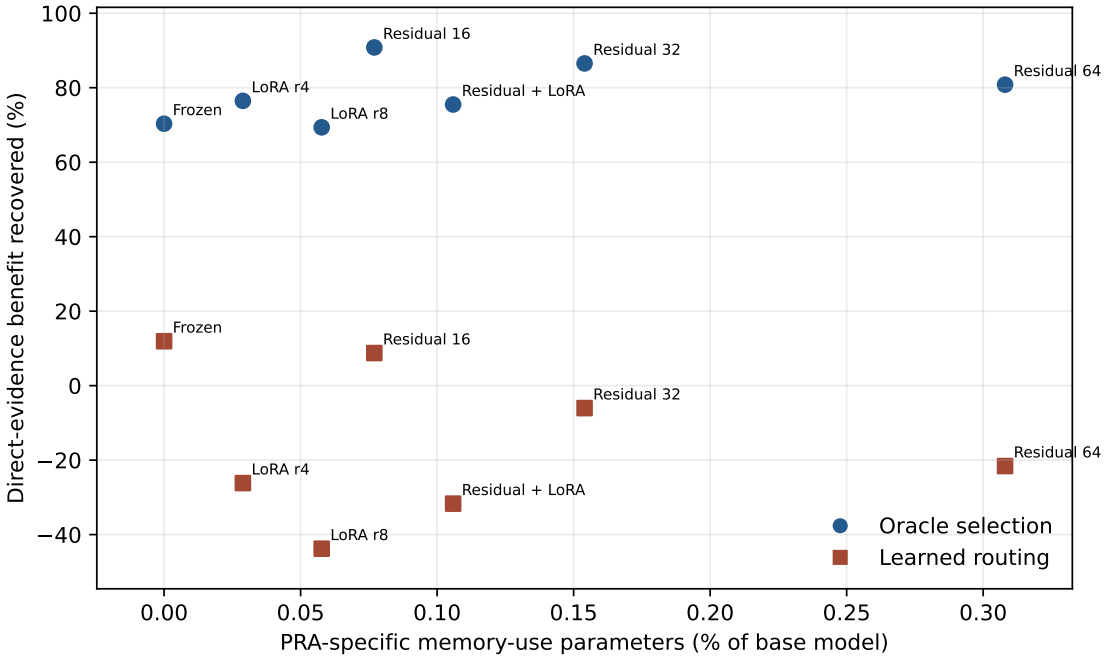

Figure 7: HotpotQA recovery versus PRA-specific memory-use parameter fraction. Additional capacity is not monotone: the smallest residual is best under oracle selection, whereas zero memory-use parameters are best under learned routing.

Table 14: Physical memory economics, averaged over the test identities. Active transfer is the selected native K/V copied for all 14 consuming layers. Peak GPU allocation includes the model and is not an incremental K/V measurement. The routing-index ratio compares compact index bytes with the complete CPU detail-K/V bank.

Dataset	Selection	Chunks %	K/V tokens	K/V %	Index/detail %	CPU detail MiB	Transfer MiB
HotpotQA	oracle	11.29	167.1	11.39	.0282	82.35	9.14
HotpotQA	routed	6.60	94.9	6.56	.0282	82.35	5.19
QASPER	oracle	4.68	192.0	4.70	.0280	245.54	10.50
QASPER	routed	2.50	96.0	2.51	.0280	245.54	5.25

The mean peak GPU allocation is 1.17–1.18 GiB for these rows. The selected HotpotQA product point transfers 5.19 MiB per example and materializes 6.56% of source K/V; frozen routed PRA recovers 11.9% of direct-evidence and 14.1% of full-context sequence-likelihood benefit there. Requested chunk fraction, realized token fraction, index overhead, off-device detail storage, and active transfer are deliberately reported separately.

Table 15: HotpotQA rank and decoding diagnostics on the larger cohort. Probability, rank, margin, sequence likelihood, and lexical answer quality are reported separately. Negative margin means the gold first token remains below the model's top token.

Condition	Gold prob.	Gold rank	Margin	Sequence logP	F1	EM
No context	.0616	55.6	-6.513	-20.192	.025	.000
Frozen PRA, routed	.0316	34.1	-6.326	-18.291	.000	.000
Residual 16, routed	.1185	121.9	-6.659	-18.797	.028	.000
LoRA 4, routed	.1354	274.9	-8.962	-24.363	.049	.000
Residual 16 + LoRA 4, routed	.1312	284.0	-9.232	-25.243	.022	.000
Residual 16, oracle ceiling	.4094	22.8	-1.780	-5.714	.314	.250
Direct evidence text	.1973	12.4	-3.112	-4.251	.257	.125
Full original context	.3018	16.3	-3.176	-6.704	.275	.250

Memory-use quality cannot be summarized by a single scalar: an intervention can raise the target token's absolute probability while worsening its rank against competing tokens or reducing the gold sequence likelihood. Frozen routed PRA improves sequence likelihood and mean first-token rank but does not improve F1/EM. Oracle residual 16 reaches EM .25 and raises the gold probability to .409, yet the mean rank is still 22.8 and the margin remains negative. Reliable decoded QA is not closed by this adaptation ladder.

9.5 A bounded LoRA sweep raises the oracle ceiling, not routed robustness

The first ladder leaves a narrow ambiguity: perhaps ranks 4 and 8, 32 updates, or the baseline learning rate underfit the conditional output map. We test that explanation without changing the last-14 placement, native K/V, router, attention budget, or oracle-memory training objective. Stage A screens ranks 4, 8, 16, and 32 for 32 and 64 updates at learning rate 10^{-3}. Stage B expands the three best ranks to 64 and 128 updates at $0.5\times$, $1\times$, and $2\times$ that rate. The score is equal-weight mean oracle sequence $\Delta \log p$ over four held-out HotpotQA and four held-out QASPER identities. Three rank-diverse finalists then run over seeds 11, 23, 37, 53, and 71. Test identities remain unopened until this choice is frozen.

The validation-only Pareto rule selects rank 32, 64 updates, and learning rate 10^{-3}: 1,376,256 parameters, or 0.231% of Qwen3-0.6B. Rank 32 improves from 11.704 at 32 updates to 12.484 at 64 in the single-seed screen, then falls to 12.096 at 128. At the baseline rate, ranks 4 and 8 instead decline when extended from 32 to 64 updates. Longer optimization is thus selectively useful rather than a general cure for under-convergence. Rank 64 is not expanded: rank 32 already fails the learned-routing product gate, so these data do not establish saturation beyond the tested rank.

Table 16: Five-seed untouched-test result for the bounded conditional-LoRA sweep. H and Q denote HotpotQA and QASPER; O/R denote oracle/routed selection. ρ_D and ρ_F are cohort recovery against direct evidence and feasible full context. F1 is oracle/routed. QASPER recovery is undefined because direct evidence lowers likelihood on this cohort.

Method	M_ψ	% base	H O	H R	H ρ_D O/R	H ρ_F O/R	H F1 O/R	Q $\Delta \log p$ O/R
Frozen PRA	0	.000	+11.212	+1.901	70.3/11.9	83.1/14.1	.102/.000	+1.444/+1.377
LoRA $r = 32$	1,376,256	.231	**+13.375**	-8.640	**83.9**/-54.2	**99.2**/-64.1	**.310**/.022	+4.641/+4.967
Residual 32 + LoRA 32	2,294,208	.385	+13.677	-8.057	85.8/-50.5	101.4/-59.7	.268/.048	**+5.062/+5.363**

The selected LoRA improves oracle HotpotQA by 2.164 ± 1.425 nats relative to frozen PRA and does so in all five seeds. Under the unchanged learned router, however, its paired effect is -10.541 ± 4.841, again with the same direction in all five seeds. Mean routed gold rank worsens from 34.1 to 124.9 and EM remains zero. QASPER routed likelihood improves by 3.590 ± 1.246, but

that cohort has no positive direct-text denominator. The result is therefore dataset-conditional, not a general routed-memory gain.

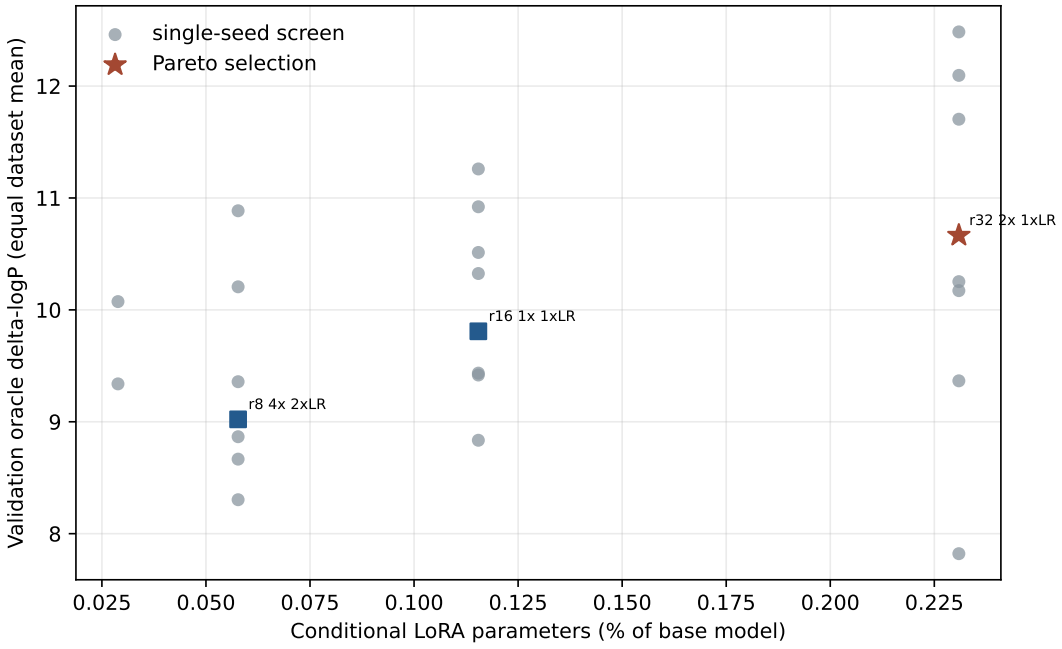

Figure 8: Held-out oracle sequence benefit versus conditional-LoRA parameter fraction. The staged screen finds an oracle frontier at rank 32 and 64 updates, but the untouched routed HotpotQA test rejects it as a product default. Parameter efficiency under clean memory does not imply robustness to imperfect selected memory.

One preregistered combination adds residual width 32 to the selected LoRA. Its combo-minus-LoRA effect changes direction across seeds for both datasets and both selection modes; the HotpotQA routed mean is only $+0.583 \pm 6.697$. We reject the combination and retain frozen last-14 PRA plus the learned router as the SDK default. The rank-32 weights are released only as a research opt-in. All 43 candidate PRA-off checks are exact, and no native-operation-limit violation occurs.

9.6 Blind behavioral-equivalence protocol

Token-level and answer-level metrics answer different questions. Logits and log-probabilities measure whether two systems follow similar probability trajectories. F1 and exact match measure lexical agreement with a reference answer. Neither establishes whether two generated answers are equally meaningful and useful to a reader. We therefore release a third, explicitly separate evaluation: a blind LLM-judge package for semantic and behavioral equivalence. This package is an evaluation protocol, not a reported judge result; no external-judge scores had been collected at the time of writing.

The package is generated deterministically from the identity-disjoint last-14 artifact. Each item contains an opaque ID, the question, and outputs A and B. It contains no model, dataset, PRA, native, routing, adaptation, or source-order labels. A private truth file maps each item back to its conditions and records model revision, generation mode and seed, realized selected-chunk

and materialized-K/V fractions, source-artifact hash, prompt hash, repository revision, and the randomization seed. SHA-256 hashes of the displayed prompt and each answer make the A/B mapping independently checkable without copying labels into the blind file.

Table 17: Released blind behavioral-judge comparisons. Counts are underlying pairs before the exact reversed-order duplicate. All source generations are greedy. Full-context controls exist only for the eight HotpotQA cases whose source fits the matched 2,048-token limit.

Comparison group	Pairs	Purpose
Native no context vs. frozen routed PRA	16	Behavioral change from sparse memory
Native direct evidence vs. frozen routed PRA	16	Gap to a concise text-context control
Native full context vs. frozen routed PRA	8	Gap to feasible dense source context
Frozen routed PRA vs. residual-16 PRA	80	Adaptation effect over 16 identities and five seeds
Identical-answer calibration	16	Expected equivalence near 100 and quality near zero
Controlled-corruption calibration	16	Detect an insensitive or inverted judge
Total	152	304 blind items after order reversal

For every pair, a seeded coin flip chooses the first A/B order and a second opaque item presents the exact reverse. The judge assigns semantic equivalence in $[0, 100]$, relative quality in $[-100, 100]$ with positive values favoring B, independent A/B validity, confidence, and a reason of at most 40 words. Identical copies and controlled truncation or contradiction provide positive and negative calibration anchors. Native-vs-native sampling and faithful-paraphrase controls are not fabricated: the source artifact contains greedy generations only and no independently authored paraphrases. Likewise, it contains one routed operating point rather than decoded 5/10/20/30% K/V-fraction sweeps.

The preregistered hypothesis is that behavioral fidelity may saturate at a lower active-K/V fraction than exact logit fidelity. The current package cannot test that fraction curve because the required decoded sweep does not yet exist. It can test whether the measured frozen and adapted outputs are behaviorally interchangeable with their native controls, and whether that conclusion survives answer-order reversal. Any future judge results must be reported by comparison group, with calibration and reversal consistency, rather than pooled into one score.

Decision. PRA-specific capacity totals 262,144 parameters (0.044% of Qwen) in the selected routed system. The oracle residual probe adds 458,976 parameters, bringing the diagnostic total to 721,120 (0.121%); the rejected combination would total 893,152 (0.150%) and occupies 2.43 MiB on disk. The broader follow-up selects a 1,376,256-parameter rank-32 LoRA by held-out oracle likelihood, but rejects it for deployment after its routed HotpotQA change falls to -8.640 nats. PRA-off logits and generation are exact for every run, and native-limit violations are zero. This result is sufficient to freeze Paper 2's transport, discovery, and bounded integration claims, while leaving robust routing-conditioned adaptation and decoded QA open. HotpotQA is a multi-hop stress and causal probe here, not a final one-shot architecture: recursive evidence gathering, in which retrieved gists drive subsequent retrieval, is deferred to Paper 2.5.

10 Cross-Family Validation

One PRA execution contract spans decoder families with materially different attention organization. We distinguish three claims. *Execution portability* means that the shared core and a thin family adapter preserve the host model's measured native semantics. *Routing portability* means that the same learned-router interface runs across families; its quality may still depend on model and domain. *Functional portability* means that routed native K/V measurably changes the frozen prediction. Qwen3-0.6B, the public SmolLM2-135M Llama-family control, and official Gemma 3 1B exercise the execution and routing interfaces; Qwen and Gemma also have functional causal probes. SmolLM2 is not a Meta Llama result; the official Meta Llama 3.2-1B checkpoint remains unmeasured.

All three routers use frozen late-layer attention-input states, one mean per 32-token parent, 128-dimensional asymmetric projections, 512 optimization steps, QASPER training, and seeds 11, 23, 37, 53, and 71. Their evidence-identity R@20% values are 0.454 ± 0.007, 0.447 ± 0.051, and 0.432 ± 0.025, respectively. The execution abstraction transfers across families; routing quality remains model- and domain-dependent.

Table 18: **Matched QASPER sparse routing is of comparable order across the three measured families, with model-dependent variance and no claim of universal routing quality.** "Exact" lists the strongest disabled-path quantities checked at the named checkpoint. QASPER R@20% is five-seed evidence-identity recall. Causal $\Delta \log p$ values use separate eight-example probes and are not directly comparable in magnitude. A dash means not measured.

Property	Qwen3-0.6B	SmolLM2-135M	Gemma3-1B-IT
Checkpoint tested	Official pinned	Public pinned Llama-family control	Official pinned
Exact PRA-off parity	Logits, hidden, greedy; cache shapes	Logits, hidden, greedy; cache shapes	Logits, hidden, greedy; cache tensors
Native K/V and cache policy	GQA, native cache	GQA, native cache	MQA, HybridCache
Host attention pattern	Full	Full	22 sliding + 4 global
PRA placement consumed	Late 8	Late 8	Global 17 and 23
QASPER identity R@20%	$.454 \pm .007$	$.447 \pm .051$	$.432 \pm .025$
Routed causal $\Delta \log p$	$+.227$	–	$+.135$
Zero native-limit violations	Yes	Yes	Yes
Wheel / public API smoke	Yes	Yes	Yes

Gemma is the structurally distinct test. Its official revision has global-capable layers 5, 11, 17, and 23, but the measured routed configuration enables and actually consumes selected memory only at layers 17 and 23; all 22 sliding-window layers remain native. On QASPER, its five-seed identity R@5/10/20/30% curve is .251/.326/.432/.514. The same router transfers more weakly to HotpotQA, with $\mathrm{AUC}_{0:30} = .196 \pm .014$, so the curve is evidence for a portable routing interface rather than equal routing quality.

Gemma therefore provides an architecture-informed memory-placement policy: PRA is admitted through the checkpoint's native global-attention structure while local sliding-window computation remains unchanged. This policy is consistent with the late-layer Qwen result, but the measured families do not establish a universal placement rule.

On eight deterministic causal-probe examples, routed Gemma memory enters layers 17 and 23 and raises mean gold-token log-probability by 0.135 nats while placing 0.060 attention mass on external memory. F1 remains 0.125, exactly equal to no memory. Forcing annotated parents into only the final global layer changes likelihood by -0.016 nats; direct evidence text changes it by $+3.765$ nats and reaches .536 F1. The measured chain is therefore routed memory $\rightarrow$ selected

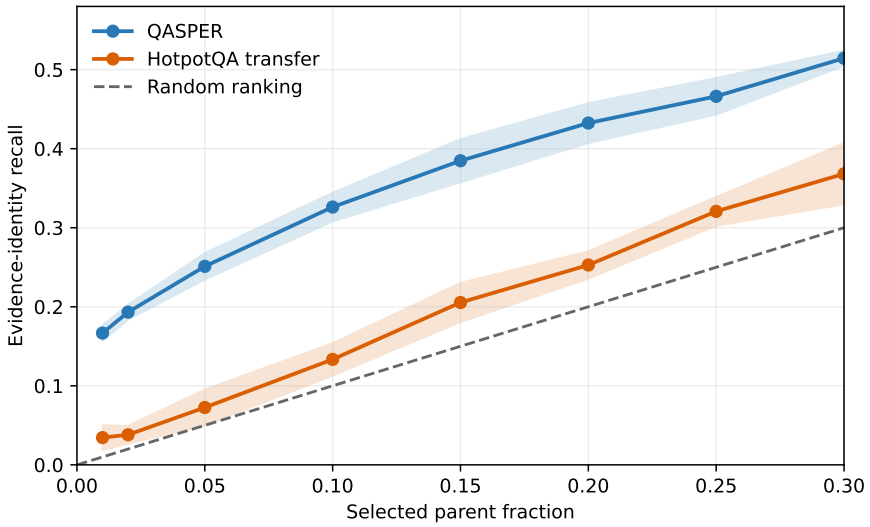

Figure 9: **Gemma routing quality is sparse-budget and domain dependent.** Five-seed evidence-identity recall is useful on QASPER, its training domain, and transfers more weakly to HotpotQA. Neither curve reaches 0.80 before exhaustive selection.

native K/V → positive gold-likelihood effect → no greedy-F1 change. It is a causal mechanism probe, not an accuracy estimate or evidence that Gemma+PRA outperforms Gemma.

10.1 Likelihood–Decoding Disconnect

Retrieved memory can raise target-token probability, improve vocabulary rank, or increase gold sequence likelihood without crossing the autoregressive argmax boundary needed to change a greedy output. Qwen supplies a large instance: the bounded-head probe gains 5.469 nats and moves the first-token rank from 94,106 to 1,351 while retaining the wrong answer. Gemma supplies an independent family-level instance: routed K/V gains 0.135 nats but leaves F1 unchanged. The cohorts and effect sizes differ, so this is not equal behavior across families. Nor is the disconnect unique to PRA; it is a general distinction between distributional improvement and decoding. Probability, rank, sequence likelihood, lexical answer metrics, and semantic equivalence are therefore separate evaluation gates.

11 Product and Systems Boundary

The validated path is packaged as `pra-hf`, with a stable Python API, CLI, serializable configuration, fraction-based selection, and Hugging Face-style router artifacts. Version `0.2.0rc1` builds as a wheel and is pinned by tag `paper2-hf-rc1`. A caller adds a reference and generates through the same model object:

```
from pra_hf import PRAConfig, PRAForCausalLM

model = PRAForCausalLM.from_pretrained(
    BASE_MODEL, routing_adapter=ROUTER,
    pra_config=PRAConfig(selected_fraction=0.20))
```

```
ref = model.add_reference_file("paper.txt")
result = model.generate(
    question,
    return_details=True,
)
model.remove_reference(ref)
```

The API also supports in-memory text, explicit URI/text pairs, chat formatting, `#__head` rollover, cache clearing, and per-request statistics. A router artifact contains projection weights, configuration, and a model card; it pins the base model revision, family, routing layer and representation, parent size, dataset splits, seeds, metrics, PRA version, and source revision without republishing base-model weights. Qwen's router has 262,144 parameters (0.044% of 596,049,920); SmolLM2's has 147,456 (0.110% of 134,515,008); Gemma's has 294,912 (0.0295% of 999,885,955). Each stores one 128-dimensional FP32 vector, 512 bytes, per parent. The repository's `pra-hf-demo/pra_hf_model_families.ipynb` exercises the same public reference lifecycle and model-family interface for Qwen, Llama-family, and Gemma users.

Memory accounting uses three denominators. *Routing-index bytes* are the compact vectors searched once per request. *Resident detail bytes* are all layer-native K/V retained on CPU. *Materialized bytes* are the selected K/V transferred for each consuming layer. A 20% requested parent fraction need not become 20% active K/V: whole-parent rounding and the hard token cap may reduce it. Conversely, replaying one selected identity through eight layers incurs eight layer-specific payloads even though semantic routing happened once.

Table 19: **The same product path runs on Qwen3-0.6B, SmolLM2-135M, and Gemma3-1B-IT with bounded materialization.** Four-example CUDA means on a GTX 950M are a systems demonstration, not a throughput or QA benchmark. Peak GPU allocation includes the full model.

Family	Evidence	F1	Requested	Materialized	Route ms	Peak GiB
Qwen3-0.6B	0.667	0.033	20.4%	6.61%	6.40	1.126
SmolLM2-135M	0.750	0.028	21.0%	6.25%	32.74	0.261
Gemma3-1B-IT	0.708	0.417	20.6%	6.48%	14.91	1.935

Qwen averages a 45.9 KiB routing index, 91.3 MiB of resident eight-layer detail K/V, and 4.0 MiB transferred per request across the eight consuming layers. SmolLM2 averages 48.8 KiB, 18.2 MiB, and 0.75 MiB, respectively. Gemma averages a 46.4 KiB index, 5.79 MiB of two-layer detail K/V, and 0.246 MiB transferred; mean TTFT/TPOT are 0.452/0.154 seconds. Mean wall time for 16 routed output tokens is 2.15 seconds for Qwen and 1.85 seconds for SmolLM2, with no native-limit violations in any family. These values describe an eager Python implementation with exact Torch search. They are not evidence of serving speedup: there is no matched dense long-context baseline, fused kernel, asynchronous transfer, or concurrency study.

The public implementation makes the tested boundary reusable. It preserves native GQA storage rather than permanently expanding K/V heads, reports both requested and actual materialization, and permits exact disabled-path testing for a new family. The release includes examples, router save/load artifacts, the model-family notebook, package and CLI tests, and a passing repository test suite. The thin Llama adapter resolves installed rotary and eager-attention functions while the shared core retains publication, routing, budgeting, and replay. Full code-level guidance appears in Appendix B.

12 Limitations and Next Experiments

The experiments establish mechanisms on modest cohorts, not benchmark accuracy. The primary representation study uses 16 matched examples, the learned router has 32 held-out test identities, and the oracle depth study uses four examples per dataset. The final adaptation study improves identity hygiene—12 HotpotQA train, four validation identities per dataset, and eight test identities per dataset—but is still small. Five seeds measure optimization variation on fixed test identities; they are not five independent dataset samples. The 1,024-token WikiText-2 control can detect drift but is not a broad language-capability evaluation.

The released behavioral-equivalence package contains 304 blinded items, but it does not yet contain external-judge responses. It therefore adds a reproducible semantic evaluation protocol, not evidence that PRA and native generations are behaviorally equivalent. Judge-model sensitivity, calibration against the identical/corrupted controls, and reversal consistency must be reported before making such a claim. A decoded K/V-fraction sweep is also required to test whether semantic fidelity saturates earlier than logit fidelity.

Evidence labels and answer behavior measure different things. Any-evidence recall can be high while a multi-hop example still lacks another required passage. Evidence-identity recall is stricter, but it still assumes that annotations identify the state this checkpoint needs. The positive HotpotQA direct-text control makes its causal comparison interpretable. QASPER direct text lowers the target likelihood on the selected causal subset, so that subset cannot grade PRA transport in isolation or define a recovery ratio. Full QASPER sources also exceed the matched 2,048-token control budget. Reliable routed decoded-QA gains remain unestablished on both datasets.

The router is domain-dependent and memory exposure is depth-sensitive. Cross-domain any-evidence recall@3 is 0.213 in both directions. Additional gist resolution, centered-RoPE features, query pooling, width, and negative-mining variants do not remove this limitation. All-layer native-K/V injection is also harmful despite using more memory. A larger study should separate calibration, model selection, and test identities. The final study does so, but its oracle-trained adapters remain brittle to routed selections: more M_ψ capacity reduces routed HotpotQA recovery. Future adaptation should train against realistic routing errors and report any-, identity-, and all-evidence recall against both selected-parent and exact-K/V fractions.

The systems result is bounded execution, not serving speed. Detail K/V remains CPU-resident, search is eager and exact, and selected rows are padded. Fused attention, approximate indexing, asynchronous transfer, batching, concurrency, and a matched dense long-context baseline remain unmeasured. CPU residency is not free: reference encoding, host storage, selection, and CPU-to-GPU transfer remain in the measured path. SmolLM2-135M is reported only as a measured Llama-family control; the official Meta Llama 3.2-1B checkpoint remains unmeasured. Gemma 3 uses the official pinned checkpoint, but its five-seed test has only 16 identities per dataset, its systems demonstration has four examples, and its causal probe has eight. Those results validate the integration boundary and causal activation, not benchmark QA quality or calibrated memory use.

The tested checkpoints establish portability through approximately one-billion-parameter scale across multiple decoder families. They do not establish 7B–70B scaling, production throughput, or the economics of a serving-optimized implementation; those require larger hardware, matched dense baselines, and fused transfer/attention measurements.

Paper 2 now stops at the measured one-shot boundary. Recursive multi-hop gathering belongs to Paper 2.5. Multi-gist strategy, semantic positional distance, routing-error-aware adaptation, and large-scale vLLM serving are separate research questions. Global LM-head tuning is not the next adaptation step: it fails to improve the memory-specific increment, changes no-PRA behavior, and

raises WikiText-2 loss.

13 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [5]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [1, 12]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [4, 13, 6]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

Prompt Cache and CacheBlend reuse modular cached state but emphasize schema-aware composition or selective recomputation [3, 11]; StreamingLLM retains a chronological working set during decoding [9]. PRA-HF instead maintains an addressable semantic index, selects whole external parent identities, and injects their exact layer-native payload under a hard active-K/V budget. General source-position ownership, retrieval-time rebinding, and pooled-RoPE geometry belong to the controlled companion study [7]; the present contribution is exact pretrained-family integration, transfer of the routing/payload separation, causal use, and a reusable interface.

14 Conclusion

PRA-HF retrofits pretrained Hugging Face decoders with external native-K/V memory while preserving their ordinary behavior exactly when disabled. Bounded encoding produces compact gists and off-path detail K/V; routing selects whole chunks for a fixed native-attention budget. The shared core works through thin Qwen, Llama-family, and Gemma 3 adapters without checkpoint conversion. Gemma's native local/global structure provides a placement prior: the measured policy leaves all sliding-window layers untouched and admits memory only through selected global layers. This is an architecture-informed compatibility result, not a universal placement rule.

The architectural result is to route semantic state and attend native positional state. Paper 1.5 explains the controlled geometry behind this separation; Paper 2 shows that the separation transfers to pretrained Qwen. Hidden-state gists avoid the positional bias of post-RoPE key means, while selection still returns the host layer's exact post-RoPE keys and native values. The learned Qwen router establishes a useful recall–sparsity frontier; matched QASPER results on SmolLM2 and Gemma show that the interface, rather than identical retrieval quality, is portable.

Selected memory measurably affects pretrained predictions, but its use is depth dependent: the HotpotQA oracle improves over a late band and degrades when replayed through every layer. Clean oracle memory also shows that tiny memory-use adapters can approach the integration ceiling. Those same adapters do not survive the routed HotpotQA distribution; on the larger disjoint test, the frozen routed path remains better. The gap is consistent with oracle training and routed deployment, but the experiments do not isolate that explanation. Routing-conditioned robustness and decoded task improvement remain open.

The released `pra_hf` wheel, family adapters, model-family notebook, router artifacts, metrics, blind-judge package, and reimplementation tests make these boundaries independently testable. PRA-HF establishes that pretrained transformer memory can be externalized, semantically routed, and reintroduced as exact native K/V under bounded operations while preserving the original model path when disabled.

A Routing Ablations

This appendix records the secondary interventions that shaped the final design. They are useful negative results: each changes one plausible source of routing error while leaving native detail K/V unchanged.

Table 20: Parameter-free diagnostics with 32-token parents. Cohorts differ as described in the artifact metadata, so rows summarize directions rather than a single paired leaderboard.

Intervention	Sparse result	Cost or control	Interpretation
Hidden segment means, $G = 1/2/4/8$	Any R@3 0.391/0.313/0.266/0.281	Index overhead 1.57/3.14/6.28/12.56%	Finer untrained means do not improve sparse ranking
Token-level hidden maximum	Any R@3 0.375 versus mean 0.438	Index overhead 50% versus 1.57%	Resolution helps some Hotpot ranks but hurts QASPER
Centered-RoPE, $G = 8$	Any R@3 0.250	Token-QK order correlation 0.874	Coordinate repair does not create evidence semantics
Question decay, strongest tested	Any R@3/MRR 0.250/0.279	Last state 0.469/0.413	More query tokens are not automatically a better query
Width 256 and harder negatives	No validation-selected gain	Twice the 128-wide index at width 256	Data and objective generalization dominate raw capacity

The matched 16-example centered-RoPE control makes the dissociation explicit. For $G = 1/2/4/8$, native token-Q/K maximum-rank correlation is .521/.602/.709/.874, while any-evidence R@3 is .188/.125/.250/.250 and routing-index overhead is 1.57/3.14/6.28/12.56% of detail K/V. More faithful token-Q/K ordering therefore does not produce monotone semantic-recall improvement. A separate fractional-center control gives R@3 .125/.125/.125 and R@16 .375/.250/.375 for exact/floor/ceil center policies. The rounding convention changes neither the direction nor the interpretation of the result. Full per-example and aggregate records are retained in `docs/papers/shared/results/paper2_hf/routing/centered_rope/`.

The chunk-size sweep supplies a related caution. At 128-token chunks and $k = 8$, recall reaches 0.688 but the selected fraction is 41.3%; at $k = 16$, 71.5% of parents are selected. Coarser chunks can improve apparent fixed-k recall by making each choice cover more source text. That is not a sparse win, which is why the main paper reports selected fractions and physical K/V.

Independent hidden-state confirmation over 64 evaluations gives any/all-evidence recall of 0.391/0.063 at $k = 3$, 0.578/0.203 at $k = 8$, and 0.797/0.500 at $k = 16$. The selected-parent fractions are 0.045, 0.119, and 0.238, while the hard materialization budget holds the latter two near 0.059 physically active K/V. This illustrates all three distinctions at once: any versus all evidence, selection versus materialization, and fixed rank versus selected fraction.

B Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit `db50846`. This checkpoint makes attention-input hidden-state routing the explicit default and exposes the fixed-selection materialization boundary. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time. The multi-layer identity mapping, experiments, tests, and complete artifacts are pinned separately at commit `8f6ef41`; all multi-layer line references below refer to that revision.

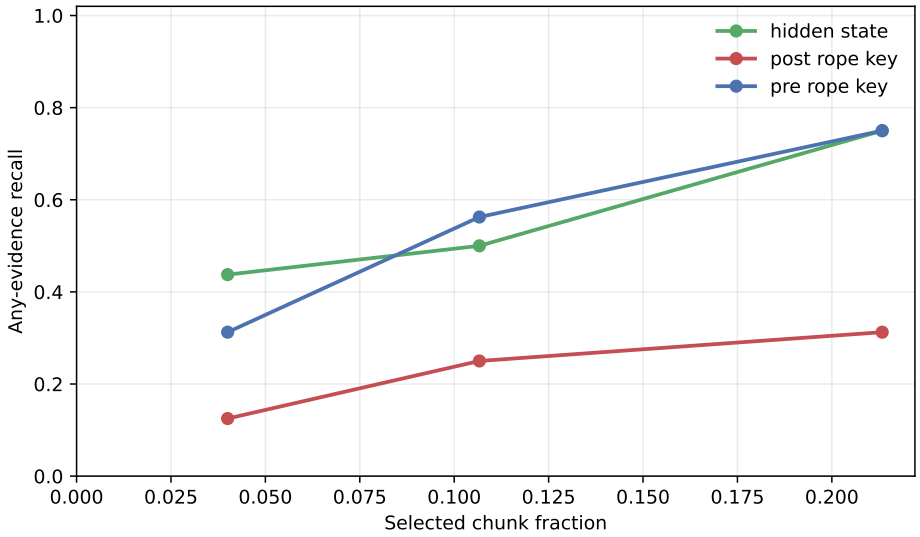

Figure 10: Qwen parameter-free routing diagnostics. Semantic hidden-state and pre-RoPE summaries dominate the post-RoPE key mean for evidence selection, while every fixed selected identity still materializes the same native post-RoPE K/V. This 16-example transfer control is smaller than the learned-router cohort and is not a paired leaderboard with it.

B.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \qquad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \tag{11}$$

Reference entries use $B = 1$ and keep H_{kv}, including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen's installed eager kernel performs the corresponding operation in the production adapter.

B.2 Minimal forward-path skeleton

The following pseudocode is the adapter's essential control flow. It is intentionally family-neutral; "native" operations must call the installed model implementation rather than reimplementing its mathematics.

```
01  forward(hidden, native_positions, native_mask, past):
02      if capture_is_armed:
03          q_pre, k_pre, v = native_project_and_norm(hidden)
04          q_post, k_post = native_position(q_pre, k_pre)
05          save_transient_routing_features(hidden, q_pre, q_post, k_pre)
06          save_post_position_detail_kv(k_post, v)
07
```

```
08      if memory_is_disabled or reference_cache_is_empty:
09          return original_attention(hidden, native_positions,
10                                    native_mask, past)
11
12      q_pre, k_pre, v = native_project_and_norm(hidden)
13      q_post, k_post = native_position(q_pre, k_pre)
14      if past is not None:
15          k_post, v = past.update(k_post, v, layer_id, native_kwargs)
16      route_q = matched_query(hidden[:, -1, :], q_pre, q_post)
17      memory = shared_core.prepare_memory(q_post, route_q,
18                                          direct_tokens=k_post.shape[2])
19      if memory.is_empty:
20          return native_attention_and_output(q_post, k_post, v, native_mask)
21      k, v, mask = shared_core.combine_local_and_memory_kv(
22          k_post, v, memory, native_mask, query_tokens=q_post.shape[2])
23      return native_attention_and_output(q_post, k, v, mask)
```

Lines 8–10 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 190–201, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 204–207 and `Cache.update` occurs once at lines 208–215. Lines 216–221 choose the matched routing query without altering the post-RoPE attention query. If routing returns no materialized memory, lines 224–234 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 394–436, pads variable memory rows, prepends memory K/V, creates an all-visible additive mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen's eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 92–110 and 246–258. Thus local and memory positions share one softmax and one output projection.

B.3 Optional learned routing metric

The learned adapter is a replaceable routing component, not part of native attention. For an asymmetric linear metric,

$$r_q = \mathrm{norm}(W_q h_q), \qquad r_c = \mathrm{norm}(W_c g_c), \qquad s(q,c) = r_q^\top r_c. \tag{12}$$

A shared adapter aliases $W_q = W_c$. An independent implementation needs only the following boundary, revision-pinned at commit `a657bc7`:

```
01  # Reference publication, after ordinary hidden-state gist pooling
02  routing_gist = adapter.project_memory(routing_gist)
03  cache.store(routing_gist, native_post_rope_k, native_v)
04
05  # Live routing; native attention continues to use q_post
06  information_need = aggregate_query_states(h_in, strategy)
07  routing_query = adapter.project_query(information_need)
08  selected_ids = cache.search(routing_query)
09  k_mem, v_mem = cache.materialize_native_kv(selected_ids)
```

```
native_attention(q_post, concat(k_mem, k_local),
                 concat(v_mem, v_local))
```

The projection implementation is `hf/routing_adapter.py`, lines 10–74; checkpoint loading and freezing are lines 77–94. Memory projection occurs at `hf/injection.py`, lines 238–246, and live query projection at `hf/qwen.py`, lines 188–202. Cast hidden features to the adapter parameter dtype at this boundary; the checkpoint is FP32 while Qwen emits FP16 states. Do not cast, project, or recompute native detail K/V. Their exact-equality invariant is the test that prevents a routing retrofit from silently becoming an attention rewrite.

B.4 Oracle selection and causal diagnostics

An oracle must replace only identity selection. Adding a separate K/V injection path would confound semantic correctness with different budgeting, positions, masks, or attention code. The operational handle activates a layer set and assigns optional row-local identities in `hf/injection.py`, lines 69–89. The Qwen adapter chooses ordinary routing or the fixed set at `hf/qwen.py`, lines 273–286; both enter `core.py::prepare_selected_memory`, lines 337–388. The latter still applies the trigger, deduplication, whole-chunk budget, overlap policy, residency transfer, and rectangular packing. The fixed-set setter and its ownership contract are in `hf/adapter_base.py`, lines 71–83.

The experiment constructs oracle identities by intersecting annotation token spans with parent logical spans in `run_oracle_memory_use.py`, lines 107–134. Lines 194–281 append the gold continuation, gather its autoregressive token probabilities, retain opt-in eager-attention weights, and use temporary decoder-layer hooks for answer-position hidden states. Attention capture is disabled immediately afterward and is never part of ordinary inference. Conditions and paired deltas are assembled at lines 310–419. A reimplementation should assert three invariants per example: requested oracle IDs equal retained IDs, budget rejection is zero, and the largest native operation stays below the declared bound.

B.5 Route-once multi-layer consumption

Multi-layer PRA shares an identity decision, not an attention tensor. An implementation should make the distinction visible in its types:

```
selected = route_once(query_at_layer_27)       # parent IDs + scores
for layer in consumption_layers:
    layer_hits = []
    for hit in selected:
        chunk = cache[hit.uri].layer[layer].by_id[hit.chunk_id]
        assert chunk.kv.was_projected_by == layer
        layer_hits.append(hit.with_payload(chunk.kv))
    adapter[layer].set_fixed_selection(layer_hits)
model(question)                                # no further semantic search
```

The operational API activates an arbitrary layer set at `hf/injection.py`, lines 70–93. Identity-to-payload resolution is lines 97–145: for each requested layer it looks up the stable `chunk_id` in that layer's `LayerReferenceMemory`, replaces the payload, and records the source routing layer. A missing layer or chunk is an error rather than a fallback to another layer's K/V. The Qwen adapter's fixed-selection branch at `hf/qwen.py`, lines 273–289 then calls the same budgeting, transfer, mask, and attention path as ordinary routing.

The experiment captures the last-layer information-need state and performs one complete ranking at `run_multilayer_pra.py`, lines 161–198. Oracle and routed identities are mapped at lines 277–325; the remainder of the function records per-layer attention, hidden-state changes, materialized tokens, transfers, and latency. The invariant test at `tests/test_hf_integration.py`, lines 548–586 asserts equal parent IDs but different target-layer objects, tensor pointers, and values while preserving native $[B, H_{kv}, T, d_h]$ shape. This catches the most dangerous superficially working implementation: copying $K^{(27)}, V^{(27)}$ into every consuming layer.

B.6 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```
01  disable_PRA_memory_during_reference_encoding()
02  for [block_start:block_end] in bounded_encoding_blocks(source):
03      position_ids = logical_source_positions(block_start, block_end)
04      arm_post_position_kv_capture(position_ids)
05      model(block_ids, position_ids=position_ids, use_cache=False)
06      captured = consume_native_kv_for_each_PRA_layer()
07      for routing_window in overlapping_windows(captured):
08          gist = pool(routing_window.attention_input_hidden_state)
09          publish(uri, logical_offsets, post_rope_k, native_v, gist)
10          discard_token_level_hidden_state()
11  restore_previous_memory_state()
```

The concrete loop is `hf/injection.py`, lines 235–354. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 266–292). Each chunk slices the captured post-RoPE K/V, chooses transient routing features through `_routing_points` (lines 172–233), computes one or more compact gists, and records logical source offsets (lines 293–348). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The contiguous multi-gist extension keeps that ownership boundary explicit. For a requested count G, split the routing feature rows into balanced source-order ranges, mean each range, score all resulting $[G, D]$ rows, reduce to one parent score, and preserve the winning local index for diagnostics. Top-k must then operate on parent identities before budgeting; otherwise several winning gists could duplicate one parent K/V payload. The reference implementation is revision-pinned at `0205df8`: pooling is in `gists/segment_mean.py`, lines 11–56, publication remains `hf/injection.py`, lines 163–190, and packed scoring plus max reduction remains `memory.py`, lines 487–593.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 357–370, the prefix is published under the reserved `#__head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

B.7 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus

output projection. Implement the six abstract methods declared at `hf/adapter_base.py`, lines 85–106, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 379–428. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 55–72).

2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 74–93).

3. Verify matched pre/post capture, representation switching, unchanged post-RoPE detail K/V, GQA grouping, serialization, and device parity (lines 109–214).

4. Supply one selected identity through router and oracle traces; require identical native K/V and attention output (lines 241–280).

5. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 216–239).

6. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 282–313).

7. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 315–330).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

B.8 Llama port and product API

The release implementation is pinned at commit `809694b`. The common abstract family contract remains in `adapter_base.py`, lines 32–132. Qwen's implementation resolves its installed helper functions through an overridable method at `qwen.py`, lines 38–77. The Llama module at `llama.py`, lines 8–48, subclasses that shared projection/capture/control path, replaces the symbol resolver, and invokes Llama's native eager function without the Qwen sliding-window argument. This is inheritance of a family-neutral tensor contract, not reuse of Qwen parameters. Injection selects Qwen or Llama from the concrete attention module at `injection.py`, lines 380–430.

An independent family port should make the same decision only after comparing installed forward signatures and testing every shared assumption. For Llama those assumptions are: Q/K/V linear outputs reshape to $[B, H, T, d_h]$; RoPE receives the same cosine/sine tuple; cache update accepts the layer index, sine, cosine, and cache position; the eager kernel repeats native K/V heads internally; and output returns token-major states before one native output projection. If any family differs, override that specific abstract method rather than adding a family branch to the router, cache, or budgeter. Tests in `test_hf_llama_integration.py`, lines 51–106, cover exact disabled behavior, native-head reference payloads, active memory, and rollover.

The public layer composes rather than replaces those internals. `pra_hf/config.py`, lines 14–127, validates stable product names, resolves negative layer indices, and translates them to the core configuration. `pra_hf/router.py`, lines 14–110, defines the versioned weights/metadata/model-card artifact and legacy-checkpoint conversion. The high-level model in `pra_hf/model.py`, lines 42–358, owns tokenizer/model loading, router compatibility, reference lifecycle, fraction selection, route-once identity replay, generation, chat, and memory economics. In particular, lines 192–258 flatten complete reference-grouped rankings, apply a global fraction or top-k cutoff, reconstruct typed identities, and map those IDs to each consuming layer's native payload. The CLI in `pra_hf/cli.py`, lines 22–145, is a thin caller of the same API; it does not maintain a second execution path. Product tests at `test_pra_hf_api.py`, lines 66–132, pin config serialization and precedence, router save/load and freezing, add/remove/clear, fraction routing, generation, statistics, and the rule that a router cannot change after gists have been indexed.

B.9 Gemma 3 hybrid-attention port

Gemma requires one additional decision before the ordinary family hooks: determine whether a layer is eligible at all. The helper in `hf/gemma3.py`, lines 8–17, reads the checkpoint's `layer_types` schedule and returns only `full_attention` indices. The adapter constructor at lines 34–60 rejects `is_sliding` modules and republishes `is_sliding=False` on the wrapper. This second assignment is necessary because the parent decoder layer inspects the child before selecting its global or local rotary embedding. Omitting it can silently send a global layer through local positional geometry even if the K/V tensors have plausible shapes.

Projection, per-head Q/K RMS normalization, native RoPE, cache update, and output projection are inherited from the already tested family-neutral path. Lines 63–71 then assert the concrete $[B, H_q, T, d_h]$ query and $[B, H_{kv}, T, d_h]$ payload layout. For Gemma 3 1B, $H_q = 4$, $H_{kv} = 1$, and $d_h = 256$. Lines 73–91 call the installed Gemma eager function with the checkpoint's scaling and logit soft cap, explicitly setting no sliding window because this adapter can exist only at a native global layer. K/V head repetition, if needed by the attention calculation, remains inside that native kernel; cached reference payloads stay one-head MQA. The injector's only family-specific addition is the concrete-class selection at `hf/injection.py`, lines 488–499.

The product layer independently enforces the same boundary. `pra_hf/config.py`, lines 65–120, maps the default final layer to Gemma's final global layer, intersects the default late band with the global schedule, and rejects explicit local-layer requests. This duplication is a public validation boundary, not a second implementation of attention. The pinned runner at `experiments/paper2_hf/gemma/run_gemma3_1b.py`, lines 134–167, audits the physical schedule, heads, RoPE bases, normalization, cache, and masks before injection. Lines 170–366 then gate exact disabled tensors, unchanged local module classes, native post-RoPE CPU K/V, continued source positions, causal-prefix invariance, finite enabled output, and bounded `#__head`. Only after those checks does the orchestration at lines 434–543 create features, train five 128-dimensional routers, exercise the public API, and run one causal probe.

The architecture suite in `tests/test_hf_gemma3_integration.py`, lines 93–262, uses Transformers' actual Gemma classes rather than a mock attention module. It pins six independent contracts: exact hybrid-cache parity; untouched local layers; rejection of invalid placement; native global RoPE and Q/K normalization; one-head post-RoPE reference K/V; and bounded product API plus runner behavior. The official-checkpoint runner adds measured pretrained routing, economics, and causal artifacts; the architecture tests remain independent exactness gates.

B.10 Memory calibration path

The calibration code preserves the same ownership boundary. The registered scalar bank in `hf/memory_gate.py`, lines 19–94, supports fixed, shared, and per-layer values. The residual bank in `hf/residual_adapter.py`, lines 10–111, lazily constructs only the requested bottleneck and enables gradients only for that width. Its zero output projection is what makes Equation 8 initially equal frozen PRA rather than a random perturbation. Lines 27–38 accept $[B, T, D]$ hidden and memory-residual tensors, deliberately discard the ordinary hidden state, and return $d_l + W_{up}\,\mathrm{SiLU}(W_{down} d_l)$ in FP32. This delta-only input prevents the adapter from learning an unrelated ordinary-state path.

The Qwen/Llama forward path computes y^{mem} first. Only when a trainable gate or residual adapter is active does `hf/qwen.py`, lines 365–412, invoke local attention a second time, form d_l, apply the gate and adapter in FP32, and cast the result back to the model dtype. The ordinary fixed-PRA path therefore does not pay for the counterfactual attention. Conditional LoRA is computed from the same pre-W_O memory features at lines 365–374 and added after the residual correction at lines 401–412. Registration and public configuration live in `hf/injection.py`, lines 98–140 and 480–495; adapters hold non-owning references so the model registers one parameter bank rather than duplicating it under every attention wrapper. `memory_use_parameters()` at lines 131–140 is the exclusive optimizer boundary for residual, LoRA, or their combination.

The final reproducible protocol is `experiments/paper2_hf/qa/run_last14_combo.py`. Lines 211–280 freeze and optimize only the selected memory-use owner; lines 302–340 build no-context, direct-text, and feasible full context controls; lines 380–488 evaluate oracle and routed memory with physical K/V accounting. Lines 503–690 form per-seed, cohort-recovery, confidence, and paired-combination summaries; lines 827–1050 enforce disjoint train/validation/test identities and validation-only selection. Unit tests in `tests/test_hf_integration.py`, lines 104–152 and 686–824, pin ownership, shape, initialization, exclusive combination gradients, and exact disabled-PRA bypass. `tests/test_hf_last14_combo.py`, lines 13–83, pins layer selection, ratio validity, combination parsing, and cohort-mean recovery.

B.11 Conditional late-band LoRA path

The LoRA implementation is deliberately separate from generic parameter-efficient fine-tuning. `hf/late_band_lora.py`, lines 10–50, defines one rectangular low-rank delta that accepts the pre-output-projection tensor $[B, T, D_a]$ and returns only a correction $[B, T, D]$. Lines 53–149 own a lazy per-layer bank, activate one rank/configuration at a time, and expose only that configuration's trainable factors. This supports Qwen3's $2048 \to 1024$ projection as well as Llama's square projection without family-specific dimensions in the LoRA class.

Registration occurs once in `hf/injection.py`, lines 467–482. Each attention wrapper holds a non-owning reference to that bank, avoiding duplicate parameter registration. Public rank switching and parameter selection are at lines 110–128. In the active Qwen/Llama path, `hf/qwen.py`, lines 348–374, first performs native attention over local plus selected K/V, flattens heads to $[B, T, D_a]$, applies the frozen W_O, and then adds Equation 9 in FP32 at lines 369–374 and 401–412. Reaching those lines already implies that memory exists. The earlier no-memory return and the disabled wrapper therefore never execute LoRA, which is the structural reason the no-PRA control is bit-exact rather than merely regularized toward parity.

The experiment entry point `run_late_band_lora.py`, lines 73–192, configures one exclusive calibration owner and trains it while evaluating a no-PRA control on every update. Lines 194–293 evaluate all four memory conditions; lines 295–353 plot the aggregate; and lines 355–547 run the fixed five-seed cohort and write raw, seed-level, and aggregate artifacts. Tests

in `test_hf_integration.py`, lines 122–152 and 739–790, pin validation, rectangular shape handling, zero initialization, parameter ownership, active gradients, and exact Qwen bypass. `test_hf_llama_integration.py`, lines 124–153, exercises the same ownership and bypass contract through the Llama-family wrapper.

B.12 Bounded LoRA search and portable artifact

The follow-up runner makes the selection boundary explicit. Constants and the Stage-A/B grid are in `run_overnight_lora_sweep.py`, lines 52–103; validation scoring and rank-diverse finalist selection are at lines 110–185. Lines 686–740 execute the single-seed screen, lines 742–782 repeat finalists over five seeds and freeze the validation choice, and lines 851–869 release training state before loading the untouched test identities. The full screen is retained below rather than reporting only its winner.

Table 21: Complete single-seed Stage-A/B conditional-LoRA screen, ranked by the equal-weight held-out oracle score. H and Q denote HotpotQA and QASPER. Stage C repeats three rank-diverse finalists over five seeds.

Stage	Rank	Updates	LR	H $\Delta \log p$	Q $\Delta \log p$	Mean
A	32	64	1.0e-03	+18.664	+6.304	+12.484
B	32	128	1.0e-03	+18.222	+5.970	+12.096
A	32	32	1.0e-03	+18.393	+5.014	+11.704
A	16	32	1.0e-03	+17.194	+5.324	+11.259
B	16	128	2.0e-03	+17.480	+4.363	+10.921
B	8	128	2.0e-03	+18.073	+3.698	+10.886
B	16	128	1.0e-03	+16.623	+4.403	+10.513
A	16	64	1.0e-03	+15.457	+5.193	+10.325
B	32	64	2.0e-03	+17.386	+3.118	+10.252
A	8	32	1.0e-03	+15.116	+5.296	+10.206
B	32	128	2.0e-03	+15.487	+4.857	+10.172
A	4	32	1.0e-03	+15.196	+4.952	+10.074
B	16	128	5.0e-04	+14.813	+4.056	+9.435
B	16	64	5.0e-04	+14.494	+4.341	+9.418
B	32	64	5.0e-04	+14.755	+3.976	+9.366
B	8	64	5.0e-04	+13.828	+4.888	+9.358
A	4	64	1.0e-03	+13.667	+5.011	+9.339
B	8	64	2.0e-03	+13.436	+4.634	+9.035
B	8	128	5.0e-04	+13.203	+4.530	+8.866
B	16	64	2.0e-03	+12.863	+4.806	+8.835
B	8	128	1.0e-03	+13.649	+3.684	+8.666
A	8	64	1.0e-03	+12.516	+4.091	+8.303
B	32	128	5.0e-04	+12.120	+3.523	+7.821

The public artifact separates memory-use weights from the router. `PRAMemoryAdapter` in `src/pra_hf/memory_adapter.py`, lines 12–132, validates the conditional-LoRA metadata, reconstructs the per-layer bank, and saves or loads only its state dictionary. The `PRAForCausalLM` constructors accept `memory_adapter` at `src/pra_hf/model.py`, lines 74–132; compatibility checks and late loading are at lines 159–207. Metadata pins the base revision, layer IDs, rank, compatible-router hash, source checkpoint hash, and the measured limitation. Because the adapter is structurally bypassed when PRA has no selected memory, loading it does not weaken disabled-path exactness. It remains an explicit research option rather than an SDK default.

B.13 Global readout control

The LM-head experiment is intentionally outside the product adapter. The class `LMHeadLoRA` in `run_lm_head_control.py`, lines 51–89, holds a non-owning reference to the frozen tied head and

registers only rectangular low-rank factors. The zero output factor makes its initial logits exact. Lines 92–124 restore the native head and final normalization before every variant; full-head controls clone the output matrix so optimization cannot mutate the tied input embeddings.

Lines 126–171 build fixed cached WikiText blocks and compute ordinary no-PRA causal loss. Lines 173–300 train oracle-memory examples and the frozen-logit KL control with sequential backward passes, which preserves the summed objective while bounding activation memory. Lines 302–405 evaluate no memory, routed memory, oracle memory, and direct text against both the current and frozen no-memory baselines. Lines 407–668 aggregate the language and QA controls and join them to the earlier gate, residual, and late-LoRA artifacts; lines 670–891 execute the common cohort. Tests in `test_hf_lm_head_control.py`, lines 9–37, pin zero-initialized LoRA equality, factor-only ownership, and exact-but-untied full-head cloning.

B.14 Revision-pinned source map

Table 22 connects the preceding control flow to the implementation.

Table 22: Source map for an independent adapter implementation.

Responsibility	File and physical lines
Abstract family boundary	`src/pra_torch/hf/adapter_base.py`, 16–106
Qwen projections, RoPE, native kernel, and output	`src/pra_torch/hf/qwen.py`, 78–110; 248–432
Llama native-symbol binding and eager invocation	`src/pra_torch/hf/llama.py`, 8–48
Gemma global-layer policy, native MQA, and eager invocation	`src/pra_torch/hf/gemma3.py`, 8–91
Matched routing/detail capture	`src/pra_torch/hf/qwen.py`, 109–244; `src/pra_torch/hf/injection.py`, 80–210
Disabled and active runtime paths	`src/pra_torch/hf/qwen.py`, 248–432
Routing query and GQA reduction	`src/pra_torch/core.py`, 92–121
Budget, deduplication, transfer, and materialization	`src/pra_torch/core.py`, 155–392
Fixed-selection identity/payload boundary	`src/pra_torch/core.py`, 335–392
Route-once layer-native identity mapping	`src/pra_torch/hf/injection.py`, 70–145
K/V and additive-mask composition	`src/pra_torch/core.py`, 394–436
Reference publication and selected-layer injection	`src/pra_torch/hf/injection.py`, 235–428
Depth schedules and causal protocol	`experiments/paper2_hf/qa/run_multilayer_pra.py`, 62–964
Memory gate and residual calibration	`src/pra_torch/hf/memory_gate.py`, 19–94; `src/pra_torch/hf/residual_adapter.py`, 10–111; `src/pra_torch/hf/qwen.py`, 365–412
Conditional output LoRA	`src/pra_torch/hf/late_band_lora.py`, 10–149; `src/pra_torch/hf/qwen.py`, 365–412
Last-14 five-seed convergence protocol	`experiments/paper2_hf/qa/run_last14_combo.py`, 211–1050
Global readout and ordinary-language control	`experiments/paper2_hf/qa/run_lm_head_control.py`, 51–891; `tests/test_hf_lm_head_control.py`, 9–37
Parity, routing, GQA, head, cache, and selection gates	`tests/test_hf_integration.py`, 80–824; `tests/test_hf_last14_combo.py`, 13–83
Stable config, router artifact, and public API	`src/pra_hf/config.py`, 14–127; `src/pra_hf/router.py`, 14–110; `src/pra_hf/model.py`, 42–358
Llama and product-facing gates	`tests/test_hf_llama_integration.py`, 51–153; `tests/test_pra_hf_api.py`, 66–132
Gemma hybrid-attention and pinned-runner gates	`tests/test_hf_gemma3_integration.py`, 93–262; `experiments/paper2_hf/gemma/run_gemma3_1b.py`, 134–543

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: `https://arxiv.org/abs/2105.03011`.

[3] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[7] Jorge Simao. Positional continuity for retrieved native-KV memory: Logical offsets, RoPE rebinding, and the limits of geometric repair. Companion Paper 1.5 manuscript, 2026.

[8] Jorge Simao. Progressive retrieval attention: Model-bounded sparse native-KV for long context and URI-addressed memory. Companion Paper 1 manuscript, 2026.

[9] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[10] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: `https://arxiv.org/abs/1809.09600`.

[11] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[12] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

[13] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.